SAPIENZA
UNIVERSITÀ DI ROMA

Automatic Design of Intelligent Systems: Applied to Chess

Ingegneria dell'Informazione, Informatica e Statistica

Corso di Laurea in Informatica

Laureando
Matt Clifford Trinidad Ramos

Relatore
Enrico Tronci

SAPIENZA
UNIVERSITÀ DI ROMA

Automatic Design of Intelligent Systems: Applied to Chess

INGEGNERIA DELL'INFORMAZIONE, INFORMATICA E STATISTICA

Corso di Laurea in Informatica

Candidate

Matt Clifford Trinidad Ramos

ID number 2048130

Thesis Advisor

Prof. Enrico Tronci

Academic Year 2025/2026

Automatic Design of Intelligent Systems:Applied to Chess
Bachelor's thesis. Sapienza – University of Rome

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: ramos.2048130@studenti.uniroma1.it

Contents

Abstract

This thesis investigates the application of a Quality-Guaranteed (QG) learning framework to the domain of chess. The objective is to design controllers that satisfy a scenario-level win-rate requirement while providing statistical guarantees on the probability of violation under uncertain and previously unseen scenarios.

The main contribution of this work is the instantiation of a QG loop that combines Evolution Strategies for optimization with Statistical Model Checking (SMC) for probabilistic verification. The framework iteratively refines candidate controllers by minimizing a scenario-wise feasibility loss and estimating, with confidence intervals, the violation probability associated with the required performance threshold.

For each sampled scenario, a finite internal experiment estimates the scenario-level win rate, which is then converted into a binary violation indicator. Sequential statistical estimation is employed to construct Wilson confidence intervals and determine whether the violation probability falls below a prescribed tolerance. The resulting guarantees are empirical and hold with high confidence under finite-sample estimation.

The proposed architecture is implemented in a modular system integrating optimization, verification, and counterexample management components. Experimental results illustrate both the strengths and limitations of applying quality-guaranteed methods to complex game environments, highlighting the trade-offs between optimization performance and statistical reliability.

Chapter 1

Introduction

Intelligent systems are increasingly deployed in domains where performance must be not only high, but also reliable and certifiable. From industrial automation to autonomous decision-making, the central challenge is no longer simply to *optimize* a controller, but to do so *under an explicit quality requirement* that can be measured, tested, and justified statistically. Traditional machine learning pipelines emphasize average performance on held-out test data; engineering practice instead demands statements such as "the system succeeds with probability at least τ and confidence $1-\delta$." Bridging this gap is the goal of *Quality-Guaranteed (QG) control*: a simulation-driven framework that integrates optimization with statistical verification to produce quantitative guarantees about a system's behavior.

This thesis instantiates the QG framework in a controlled but non-trivial domain: the game of **chess**. Chess provides:

- fully specified and deterministic dynamics (legal moves, terminal states);
- a rich state space that challenges simple, handcrafted policies;
- a fast and reproducible simulator, ideally suited for large-scale Monte Carlo experimentation.

Although chess carries none of the risks associated with cyber-physical systems, it is complex enough to stress-test the methodology in an adversarial and combinatorial setting. More precisely, the scenario seed fixes the high-level configuration (opponent type and initial randomization), while independent realizations are obtained by consuming different segments of the underlying pseudo-random streams (e.g., exploration choices and tie-breaking), so that repeated games under the same scenario can be modeled as independent Bernoulli trials, interpreted as a standard Monte Carlo modeling approximation rather than a formal independence guarantee.

Simulation-based design with quality guarantees

The simulation-based design paradigm assumes that:

1. the closed-loop system can be *simulated* under any choice of controller parameters;

2. executions can be *evaluated* with respect to a requirement of interest;
3. executions can be *repeated* under controlled randomization to estimate relevant probabilities.

On top of this simulation layer, QG control alternates two components:

- a **black-box optimization phase** (in our case, Evolution Strategies) that searches for controller parameters θ minimizing a requirement-driven feasibility loss;
- a **Statistical Model Checking (SMC) phase** that estimates—with confidence intervals—how often the requirement is violated under randomized scenarios.

These two phases form a closed-loop process: if SMC detects that the controller violates the requirement too frequently, the violating scenarios are transformed into *counterexamples* and added to the training set, focusing subsequent optimization precisely where the current controller fails.

Why guarantees matter

Self-play reinforcement learning systems can achieve high empirical win-rates, but without explicit limits on the probability of failure [10, 12]. In many applications, stakeholders are not satisfied with empirical averages; they require statements such as

$$\Pr[\text{success}] \geq \tau \quad \text{with confidence } 1-\delta,$$

or equivalently, letting $\varepsilon := 1-\tau$,

$$\Pr[\text{failure}] \leq \varepsilon \quad \text{with confidence } 1-\delta.$$

The QG approach is tailored to such statements, combining learning with probabilistic evidence obtained through controlled Monte Carlo experiments.

A minimal numerical example illustrates the idea. If the requirement is "win-rate at least 0.85", then an SMC estimate might report

$$\widehat{p} = 0.87, \qquad [\ell, u] = [0.86, 0.90],$$

showing that, with the specified confidence, the true win-rate exceeds the threshold.

Problem formulation in the chess domain

The chess instantiation of the QG methodology consists of:

- **System dynamics.** A deterministic chess engine acting as the environment: it generates legal moves, updates the game state, and determines terminal outcomes.
- **Controller.** A lightweight feed-forward neural network f_θ that evaluates positions and selects moves using a greedy or ε-greedy policy.

- **Scenarios.** Each scenario $s = (\sigma, o)$ fixes a random seed σ and an opponent policy o. Scenarios are reproducible and capture the variability the controller must handle.
- **Requirement.** A per-scenario win-rate threshold: the controller must achieve for all training scenarios.
$$\mathrm{WR}(\theta, s) \geq \tau$$
- **Feasibility loss.** A requirement-centric loss $J_S(\theta)$, aggregating by how much each scenario violates the threshold. In the remainder of this thesis, we denote the scenario-wise win-rate by $p(\theta, s)$, and use $\mathrm{WR}(\theta, s) \equiv p(\theta, s)$ interchangeably.

The QG workflow in brief

Overall, the QG loop applied to chess consists of four steps:

1. **Optimize (ES).** Evolution Strategies minimize the feasibility loss over the scenario set S, producing a candidate controller $\theta^\star$.
2. **Verify (SMC).** Statistical Model Checking estimates the violation probability
$$\gamma(\theta^\star) = \Pr_{s \sim D}\big[p(\theta^\star, s) < \tau_{\text{verify}}\big],$$
where $p(\theta, s)$ denotes the scenario-wise win rate and D is the distribution of unseen scenarios. A confidence interval for $\gamma(\theta^\star)$ is computed using the Wilson method.
3. **Focus (counterexamples).** All scenarios causing violations are added to the scenario set S.
4. **Iterate.** The loop repeats until the estimated violation probability falls below a prescribed tolerance $\varepsilon_{\text{viol}}$. In practice, the SMC procedure estimates an empirical proxy $\gamma_{n_{\text{int}}}(\theta^\star)$, where the scenario-level win-rate is approximated using a finite number n_{int} of internal games per scenario.

This design embodies the principle of *learning under constraints*: optimization is guided by a requirement, not merely by average reward.

Scope and assumptions

The scope of the thesis is limited by the following modeling assumptions:

- **Controller class.** A compact feed-forward network, chosen for transparency, speed, and reproducibility.
- **Scenario distribution.** Opponents include random and greedy-material policies, optionally with noise; seeds induce diversity in openings and midgame tactics.

- **Simulation budget.** The number of games per scenario and the maximum number of plies act as computational constraints.
- **Interpretation of guarantees.** All guarantees refer to the chosen scenario distribution; changing the distribution changes the statement being certified.

Positioning with respect to existing work

The thesis lies at the intersection of neural-network-based decision making, black-box optimization, and statistical verification.

Unlike modern RL/self-play pipelines—which optimize for expected performance without confidence guarantees—the proposed approach produces both a trained controller and a quantitative upper bound on its failure probability.

Unlike analytical control techniques, the method leverages simulation to handle the combinatorial dynamics of chess.

This thesis provides an end-to-end instantiation of the Quality-Guaranteed framework applied to a chess simulator, integrating training, verification and SMC in a unified pipeline.

Motivation

Reliable controller design with explicit probabilistic guarantees is increasingly important in high-reliability settings. Classical reinforcement learning methods rarely provide such guarantees, and their training objectives do not directly align with requirement satisfaction. The QG framework addresses this by embedding verification *within* the learning process rather than applying it post hoc.

Research questions

The thesis investigates:

- **RQ1.** Can a lightweight neural controller trained via Evolution Strategies satisfy a must-win requirement across a finite set of scenarios?
- **RQ2.** Can sequential verification and Statistical Model Checking provide meaningful confidence-controlled guarantees within realistic simulation budgets?
- **RQ3.** Does counterexample-guided refinement reduce the estimated violation probability and improve robustness to unseen scenarios?

Contributions

This thesis makes the following contributions:

- **QG instantiation in a discrete adversarial domain.**
- **ES-based feasibility training** tailored to non-differentiable win-rate requirements.

- **Sequential verification and SMC with absolute-error guarantees.**
- **An integrated, CPU-only software pipeline** connecting simulation, ES, verification, and counterexample refinement.
- **An empirical study** demonstrating QG behavior in the chess domain.

Structure of the thesis

The remainder of the thesis is organized as follows.
Chapter 2 introduces the chess environment, neural controllers, black-box optimization, and the statistical tools used in the work.
Chapter 3 formalizes the QG learning loop.
Chapter 4 describes the Python implementation.
Chapter 5 presents experimental results.
Chapter 6 summarizes the findings and outlines directions for future work.

Chapter 2

Background

This chapter collects the theoretical and methodological ingredients required for the rest of the thesis and connects them to their concrete realization in the Python codebase. We first introduce chess as a controlled simulation environment and discuss how game states and legal moves are represented. We then describe the neural-network controller, review basic ideas in black-box optimization and Evolution Strategies (ES), and recall the probabilistic tools used to evaluate performance. Finally, we present sequential estimation, Statistical Model Checking (SMC), and the Quality-Guaranteed (QG) learning loop at an informal level, preparing the ground for the formal treatment in Chapter 3.

2.1 Chess as a controlled environment

Chess is a two-player, turn-based, deterministic game with perfect information. At any time during a game, the full state is observable: the arrangement of all pieces on the board, the player to move, castling rights, possible *en passant* captures, and move counters. There is no hidden information and, once the rules are fixed, no exogenous source of randomness.

From a control-design perspective, it is convenient to treat chess as a *simulator*. Given a current state and a chosen move, the simulator returns the successor state by applying the rules of the game; given a state, it can determine whether the game has ended and, if so, with which outcome (win, draw, or loss). This viewpoint abstracts away from human play and focuses on the underlying state-transition relation defined by the rules.

The case study in this thesis uses chess as a testbed for simulation-based design with quality guarantees for three main reasons:

- **Deterministic dynamics.** For a fixed initial position and fixed sequence of moves, the resulting game is uniquely determined. This makes simulation conceptually simple and avoids modeling stochastic environments.
- **Rich combinatorial structure.** The branching factor and game length generate a huge state space, challenging naive or handcrafted controllers.

- **Fast software simulation.** A lightweight engine can simulate a large number of games in reasonable time on a standard CPU, which is crucial for Monte Carlo reasoning and QG verification.

The Python implementation encapsulates the game logic in a dedicated engine module. Internally, each game is represented as a sequence of states and moves, but for the QG methodology only the final outcome is needed when computing win-rates and violation probabilities.

2.2 State representation and legal-move generation

A chess *position* encodes all information required to continue the game according to the rules. In the implementation, a position object stores:

- the occupancy of each square on the 8×8 board (piece type or empty);
- the side to move;
- castling rights for both colors;
- an optional *en passant* file;
- half-move and full-move counters.

Many internal representations are possible (bitboards, arrays of pieces, mailbox encodings). The project adopts a simple but explicit structure that favors clarity and ease of debugging over raw performance. The only requirement imposed by the QG framework is that the representation be deterministic and efficiently convertible into the feature vector consumed by the neural controller.

Legal-move generation follows a standard two-step strategy:

1. **Pseudo-legal moves.** For each piece of the side to move, the engine first generates moves that obey the piece-specific movement rules (rook, bishop, knight, queen, king and pawn) but temporarily ignore king safety.
2. **Legality filter.** For every pseudo-legal move, the engine simulates the move on a copy of the position and checks whether the moving side's king would remain in check. Only moves that preserve king safety are retained as legal.

Special rules are handled explicitly:

- **Castling** is permitted only when the king and rook have not moved, the squares between them are empty, and the king does not move through or into check.
- ***En passant*** captures are available only immediately after an opposing pawn has advanced two squares and only from appropriate neighboring pawns.
- **Promotions** are encoded as moves that include the promoted piece type; all legal promotion options are generated.

Termination conditions (checkmate, stalemate, insufficient material, repetition, and the fifty-move rule) are evaluated after each move. This guarantees that all simulated games end in finite time, either by reaching a rule-based terminal condition or by hitting a maximum number of plies specified in configuration.

2.3 Neural-network-based controllers

The controller in this thesis is a neural network that evaluates chess positions and induces a policy for move selection. Formally, the controller is a function

$$f_\theta : \mathbb{R}^d \to \mathbb{R},$$

where $x \in \mathbb{R}^d$ is a feature vector encoding the current position and θ denotes all trainable weights and biases.

The encoding stage bridges the gap between the engine's internal position representation and the numerical input expected by the network. A typical encoding pipeline is:

1. **Piece planes.** For each piece type and color, a binary 8×8 plane marks the squares occupied by that piece.
2. **Global features.** Additional components encode the side to move, castling rights, and move counters.
3. **Flattening.** All planes and scalar features are concatenated into a fixed-dimensional vector x.

The network itself is a compact feed-forward architecture with one or more hidden layers and non-linear activations (e.g. ReLU). Its scalar output is interpreted as a score for the input position: higher values indicate positions the controller considers more promising for the side to move.

During match simulation the controller defines a policy:

1. the engine enumerates all legal moves from the current position;
2. for each move, the engine applies it virtually to obtain the successor state and encodes that state into x;
3. the controller computes a score $f_\theta(x)$ for each successor;
4. the move with the highest score is selected, possibly replaced with a random move with probability ε (an ε-greedy exploration mechanism [12]).

This design keeps the controller relatively simple while being expressive enough to interact meaningfully with the QG loop and the ES optimizer.

2.4 Black-box optimization and Evolution Strategies

The QG framework treats controller design as a *black-box optimization* problem. Given a parameter vector θ and a simulator, we can evaluate an objective function $J(\theta)$—in our case, a feasibility loss defined in terms of win-rates—but we do not assume access to analytical gradients or to a closed-form expression for J.

Evolution Strategies (ES) form a family of derivative-free algorithms well suited to such settings; see, for instance, [7] for a modern, scalable Evolution Strategies approach applied to reinforcement learning settings. In their simplest form, ES approximate gradient descent by probing the objective around the current parameters with random perturbations:

1. sample perturbations $\{\varepsilon_i\}_{i=1}^{\lambda}$ from a multivariate normal distribution $\mathcal{N}(0, \sigma^2 I)$;
2. construct candidate parameters $\theta_i = \theta + \varepsilon_i$;
3. evaluate the objective $J(\theta_i)$ for each candidate via simulation;
4. rank or weight candidates based on their objective values, giving more weight to lower-loss individuals;
5. update the base parameters as

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{\lambda} w_i \varepsilon_i,$$

where α is a learning rate and w_i are normalized selection weights.

Viewed in expectation, this procedure performs a stochastic gradient step on a smoothed version of J. ES are attractive for this thesis because:

- they can optimize non-differentiable objectives, such as losses involving positive parts or indicator functions;
- they naturally exploit parallelism, since different candidates and scenarios can be evaluated independently;
- they treat the simulator as a black box, requiring only function evaluations.

Chapter 3 builds on these ideas to define the requirement-driven feasibility loss $J_S(\theta)$ and its ES-based optimization.

2.5 Probabilistic performance metrics

Because chess simulations are deterministic for fixed seeds and policies, randomness arises from pseudo-randomization and from *scenario sampling*. Intuitively, each scenario corresponds to a fixed seed and opponent policy, and the controller must perform well across such scenarios.

For a given controller θ and scenario s, one match outcome is modeled as a Bernoulli random variable

$$Y(\theta, s) \in \{0, 1\},$$

where $Y = 1$ denotes a win and $Y = 0$ denotes a draw or loss (draws are counted as failures to enforce a must-win requirement). The unknown success probability

$$p(\theta, s) = \Pr\big(Y(\theta, s) = 1\big)$$

is the *true scenario-level win-rate*. The quantity $p(\theta, s)$ is not directly observable and is estimated through Monte Carlo simulation.

If n independent games are played under the same scenario, the empirical win-rate is

$$\widehat{p}(\theta, s) = \frac{1}{n} \sum_{i=1}^{n} Y_i(\theta, s),$$

where $Y_1(\theta, s), \ldots, Y_n(\theta, s)$ are treated as i.i.d. samples of $Y(\theta, s)$. Under this modeling assumption, $\widehat{p}(\theta, s)$ is an unbiased estimator of $p(\theta, s)$.

Although chess simulations are deterministic for fixed seeds and policies, repeated match outcomes under the same scenario are treated as random variables. Independent realizations are obtained by consuming different portions of the underlying pseudo-random streams (e.g., exploration choices and tie-breaking), which motivates modeling repeated outcomes under a fixed scenario as i.i.d. Bernoulli trials. This independence assumption should be understood as a modeling abstraction induced by pseudo-randomization rather than as a physical source of randomness.

The QG methodology uses these quantities in two complementary ways:

- **Local view.** On a finite training set S of scenarios, the requirement is that the true scenario-level win-rate $p(\theta, s)$ should exceed a threshold τ for all $s \in S$.
- **Global view.** Under a distribution D over scenarios, the central quantity is the *violation probability*

  $$\gamma(\theta) = \Pr_{s \sim D}\big[p(\theta, s) < \tau\big],$$

  that is, the probability that the requirement is not met.

The feasibility loss used during ES training aggregates, over the training set S, how far each empirical win-rate $\widehat{p}(\theta, s)$ falls below the threshold τ. Verification procedures then attach confidence intervals to both scenario-level win-rates and the global violation probability.

2.6 Sequential estimation and stopping rules

Estimating a binomial probability such as $p(\theta, s)$ with a fixed, large number of samples can be wasteful when the true probability is either very high or very low. *Sequential* procedures address this by adaptively deciding how many samples to collect.

For each scenario s, the verifier maintains the current number of games n_s, the number of wins k_s, and the empirical win-rate $\widehat{p}_s = k_s/n_s$. After each batch of games, a confidence interval $[\ell_s, u_s]$ is computed for $p(\theta, s)$; the interval *radius*

$$r_s = \frac{u_s - \ell_s}{2}$$

summarizes the current level of uncertainty.

The implementation uses Wilson score intervals for binomial proportions, which provide good finite-sample coverage and behave well near probabilities 0 and 1 [14]. Sequential estimation follows the classical principles of sequential analysis [13].

- Games are added in batches until either r_s falls below a target accuracy ε or a maximum budget $n_{\max}$ is reached.
- Once the stopping condition is met, the scenario is classified as: *pass* if $\ell_s \geq \tau$, *fail* if $u_s < \tau$, and *borderline* otherwise (i.e., when $[\ell_s, u_s]$ straddles τ).

This approach balances statistical accuracy and simulation cost: easy scenarios (very high or very low win-rates) require relatively few games, while borderline scenarios automatically receive more attention. In the QG loop, borderline/fail outcomes are treated conservatively as potential violations and can be used to drive counterexample-guided refinement.

2.7 Statistical Model Checking

Classical model checking aims to verify whether a system satisfies a formal specification for all possible executions. For complex or stochastic systems, exhaustive exploration of the state space may be infeasible. Statistical Model Checking (SMC) replaces exhaustive exploration with Monte Carlo sampling: it estimates the probability that a property holds by observing a finite number of simulated runs; see, for example, [1, 2, 9, 15].

In a typical SMC setting:

- a system model and a property of interest are given;
- independent runs of the system are simulated under randomly sampled conditions;
- each run is labeled as *success* or *failure* depending on whether the property is satisfied;
- the empirical frequency of success is used to construct estimators and confidence intervals for the true satisfaction probability.

In this thesis, SMC is used in a specialized way. The property of interest is a win-rate requirement: "the controller achieves scenario-level win-rate of at least τ". For each randomly drawn scenario s, a small internal experiment produces an

estimate $\widehat{p}(\theta, s)$ of the scenario-level win rate. This is then converted into a binary *violation indicator* defined as

$$Z = \begin{cases} 1 & \text{if } \widehat{p}(\theta, s) < \tau_{\text{verify}}, \\ 0 & \text{otherwise.} \end{cases}$$

The sequence of violation indicators behaves as i.i.d. Bernoulli samples with mean

$$\gamma_{n_{\text{int}}}(\theta) = \Pr\big(\widehat{p}(\theta, s) < \tau_{\text{verify}}\big),$$

which constitutes an empirical approximation of the true violation probability $\gamma(\theta)$ defined in Section 2.5. SMC is thus used to estimate $\gamma_{n_{\text{int}}}(\theta)$ with an absolute-error guarantee and a specified confidence level.

2.8 The Quality-Guaranteed learning loop

The concepts introduced so far combine into the Quality-Guaranteed learning loop, a design pattern that couples optimization and statistical verification and has been studied in previous work on QG control and verification [3].

At a high level, the QG loop alternates between two phases:

- **Optimization phase.** A candidate controller is trained on a finite scenario set S by minimizing a feasibility loss built from scenario-wise win-rates. In this thesis, ES perform the optimization, treating the simulator as a black box.
- **Verification phase.** The same controller is then analyzed statistically: sequential scenario-wise verification refines win-rate estimates on S, while SMC estimates the global violation probability $\gamma(\theta)$ under a broader scenario distribution.

Whenever verification reveals scenarios that violate the requirement, these are recorded as *counterexamples* and added to the training set. Over successive iterations, this feedback mechanism enriches S with challenging scenarios and forces the controller to improve exactly where it currently fails.

This chapter provides the conceptual and probabilistic background for this paradigm. Chapter 3 transforms the QG loop into a precise mathematical formulation, defines the feasibility loss explicitly, and presents the full algorithmic realization used in the chess application. The formal definitions of the local and global requirements, the feasibility loss, and the violation probability are given in Sections 3.2 and 3.3.

Chapter 3

Methods

Building on the conceptual background of Chapter 2, this chapter describes how the Quality-Guaranteed (QG) methodology is instantiated in the concrete chess case study implemented in Python. Rather than presenting Quality-Guaranteed control only in abstract terms, the goal here is to connect each mathematical object to the corresponding ingredients of the software pipeline: match simulation, win-rate estimators, Evolution Strategies (ES), sequential verification, and Statistical Model Checking (SMC). All algorithms presented below have direct counterparts in the Python modules `chess_engine.py` and `qg_runner.py`, together with the optimization and verification modules documented in Section 3.7. The overall structure follows the general paradigm of simulation-based design with formal quality guarantees, adapted here to a neural-network chess controller [3].

3.1 System Overview

At a high level, the system implements a closed loop that repeatedly plays chess games between a *trainable controller* (the neural-network player under design) and a family of fixed opponent policies. All experiments are driven by pseudo-random seeds, so that each simulation is reproducible once the seed and opponent type are known.

The main components are:

- a **chess engine** that maintains game states and generates legal moves;
- a **neural controller** f_θ, implemented by the `NNPlayer` class, that evaluates positions and chooses moves for the designed player;
- an **opponent library** containing simple, fixed policies `Random-Player`, with the full list of opponents given in Section 3.7;
- a **scenario manager**, which defines and stores the scenarios used both for training and for verification;
- an **ES optimizer** that searches over the parameter vector θ to reduce a feasibility loss;

- a **verification layer**, which provides scenario-wise sequential verification and global SMC.

A single chess match is generated as follows. The scenario manager provides a scenario $s = (\sigma, o)$, consisting of a seed σ and an opponent policy o. The seed is used to initialize all pseudo-random generators (for example, for ε-greedy exploration), and the opponent policy describes how the other player selects moves. The engine starts from a fixed initial position (or, in some experiments, from a pre-defined midgame position) and alternates between:

- querying the controller when it is the designed player's turn
- querying the opponent policy when it is the other side's turn.

The state is updated until the engine detects a terminal condition (win, draw, or loss). The final result is mapped to a numerical *match outcome*, which is then aggregated across multiple games to form empirical win-rates.

The QG methodology is embedded in this loop as follows:

1. given a finite set of training scenarios S, the ES optimizer uses batches of simulated games to minimize a *feasibility loss* $J_S(\theta)$;
2. when ES has converged (or after a fixed budget of iterations), the verification layer estimates scenario-level win-rates and constructs confidence intervals using a sequential procedure based on Wilson score intervals [14];
3. in parallel, an SMC procedure estimates the *violation probability* under a scenario distribution D, again with absolute-error guarantees, following standard SMC treatments [1, 2, 9, 15];
4. scenarios that exhibit violations are stored as counterexamples and added to S, and the loop continues until the estimated violation probability is below a prescribed tolerance.

Algorithm 1 summarizes how a single match is simulated under a controller θ and a scenario s. In the implementation, this logic is realized by the match-simulation routines in `chess_engine.py` and orchestrated by `qg_runner.py`.

The remainder of the chapter makes this description precise: we formalize scenarios and requirements, explain how win-rates are modeled as Bernoulli quantities, define the feasibility loss, and describe the ES, verification, and SMC procedures used in the implementation.

3.2 Formalization of Scenarios, Win-rates, and Requirements

We begin by formalizing how empirical win-rates are computed under a fixed controller and a fixed scenario. This basic operation is used throughout ES training, sequential verification, and SMC.

1. *Scenario selection.* The scenario manager samples a scenario $s = (\sigma, o)$ either from the finite training set S or from the scenario distribution D (for SMC runs).
2. *Match simulation.* Given (θ, s), the engine repeatedly calls the controller and the opponent policy to play a complete chess game, returning a Bernoulli outcome $Y \in \{0, 1\}$ from the controller's perspective.
3. *Win-rate estimation.* By repeating match simulation n times under the same scenario, the system computes the empirical win-rate $\widehat{p}(\theta, s)$ and, when needed, a Wilson confidence interval for $p(\theta, s)$.
4. *Loss evaluation (ES).* During training, empirical win-rates over all scenarios in S are aggregated into the feasibility loss $J_S(\theta)$, which drives the ES update.
5. *Verification.* After ES has converged, scenario-wise sequential verification refines the estimates of $\widehat{p}(\theta, s)$, while SMC estimates the global violation probability $\gamma(\theta)$ under D.
6. *Counterexample update.* Scenarios that violate the requirement are stored as counterexamples and merged back into S, closing the Quality-Guaranteed loop.

Figure 3.1. Logical pipeline of the Quality-Guaranteed loop.

ID	Seed σ	Opponent policy o
s_1	123456789	Random policy
s_2	987654321	Greedy-material
s_3	424242424242	Greedy-material with noise
s_4	202520262027	Random policy (different opening bias)

Table 3.1. Example scenarios used in the Quality-Guaranteed loop. Each scenario fixes a seed σ and an opponent policy o, making simulations reproducible by construction.

Example scenarios. To make the abstract definition of scenarios more concrete, Table 3.1 reports a few representative instances used in the experiments. Each scenario fixes both a base pseudo-random seed and the opponent type, making the experimental procedure reproducible. Repeated matches within the same scenario are obtained by using different derived random seeds (e.g., deterministically derived from the base seed) or by consuming different portions of the underlying pseudo-random streams, so that outcomes vary across repetitions while remaining reproducible.

Win-rates as Bernoulli quantities

From an implementation standpoint, a single chess game is represented by a match object containing the scenario identifier, the sequence of moves, and the final outcome. For Quality-Guaranteed control, only the outcome is required. Each match is viewed as a Bernoulli trial from the perspective of the designed player.

Given the controller parameters θ and a scenario s, one game produces a random

variable

$$Y_i(\theta, s) \sim \text{Bernoulli}(p(\theta, s)),$$
$$\Pr(Y_i(\theta, s) = 1) = p(\theta, s),$$
$$\Pr(Y_i(\theta, s) = 0) = 1 - p(\theta, s).$$

In the implementation, draws are treated as failures in order to enforce a must-win requirement. The underlying, unknown success probability

$$p(\theta, s) = \Pr\big(Y(\theta, s) = 1\big)$$

is the *scenario-level win-rate*. In code, $p(\theta, s)$ is never computed analytically; instead, the simulator generates repeated match outcomes under (θ, s) and records them as a sequence of zeros and ones. Although the game dynamics are deterministic for fixed seeds and policies, independence is approximated by running repeated games with different derived pseudo-random seeds under the same scenario definition.

If n games are played under the same scenario s, the empirical win-rate

$$\widehat{p}(\theta, s) = \frac{1}{n} \sum_{i=1}^{n} Y_i(\theta, s)$$

is implemented simply as the average of the collected outcomes. Under the ideal assumption that the outcomes $\{Y_i(\theta, s)\}_{i=1}^{n}$ are independent Bernoulli trials with parameter $p(\theta, s)$, this sample mean is an unbiased estimator of $p(\theta, s)$. In our implementation, independence is only approximated via derived pseudo-random seeds; therefore we use $\widehat{p}(\theta, s)$ as a standard empirical estimator and quantify its uncertainty through binomial confidence intervals.

Scenarios and requirements

A *scenario* is defined as a pair

$$s = (\sigma, o),$$

where σ is a 64-bit integer seed and o is an opponent identifier. In the software, scenarios are stored as small records containing the seed and an enumeration indicating the opponent policy (for example, random, greedy, or greedy-with-noise). Fixing s determines both the randomness of the controller (through the seed) and the behavior of the opponent.

Two levels of requirements are considered:

- a **training requirement** on a finite training set S, which demands that the win-rate exceed a threshold τ_{train} scenario by scenario;
- a **verification requirement** expressed in terms of the violation probability under a distribution D over scenarios, defined using a (possibly stricter) threshold τ_{verify}.

Formally, the training requirement reads

$$p(\theta, s) \geq \tau_{\text{train}} \qquad \text{for all } s \in S, \tag{3.1}$$

The global property is expressed in terms of the *violation probability*

$$\gamma(\theta) = \Pr_{s\sim D}\big[p(\theta, s) < \tau_{\text{verify}}\big], \tag{3.2}$$

where D is a probability distribution over scenario space (in practice, a uniform distribution over seeds combined with a discrete distribution over opponent types).

In this case study we use $\tau_{\text{train}} = 0.5$ for ES-based optimization guidance and $\tau_{\text{verify}} = 0.6$ for sequential verification and SMC-based generalization assessment.

Empirical violation probability. In practice the true win-rate $p(\theta, s)$ is unknown and can only be estimated through a finite number of simulated games. For SMC, for each sampled scenario s we compute an empirical win-rate $\widehat{p}_{n_{\text{int}}}(\theta, s)$ based on n_{int} internal games.

Hence, SMC estimates the empirical violation probability

$$\gamma_{n_{\text{int}}}(\theta) = \Pr_{s\sim D}\big[\widehat{p}_{n_{\text{int}}}(\theta, s) < \tau_{\text{verify}}\big]. \tag{3.3}$$

The QG requirement is that

$$\gamma(\theta) \le \varepsilon_{\text{viol}},$$

for a small tolerance $\varepsilon_{\text{viol}}$ (for example, 0.05), with explicit (ε, δ) guarantees on the accuracy of the estimators used for both local and global quantities. The specific numerical settings adopted in the experiments are reported in Chapter 5.

3.3 Feasibility Loss

Training is driven by a feasibility loss that penalizes shortfalls with respect to the scenario-wise requirement. For each $s \in S$, the empirical win-rate $\widehat{p}(\theta, s)$ is estimated by running a fixed number of games (specified in configuration) with controller parameters θ. The loss is defined as

$$J_S(\theta) = \sum_{s\in S} \max\big(0, \tau_{\text{train}} - \widehat{p}(\theta, s)\big). \tag{3.4}$$

Algorithm 3 corresponds to the loss computation routines used during ES training, implemented in `es_bbo.py` and invoked by driver scripts such as `run_es_once.py` and `qg_runner.py`.

By construction, $J_S(\theta) = 0$ if and only if the estimated win-rate meets or exceeds τ_{train} on every scenario in S. The implementation computes (3.4) directly from simulation results: for each scenario, it aggregates the match outcomes, computes the empirical win-rate, and adds the corresponding positive part $\max(0, \tau - \widehat{p}_s)$ to the total.

This loss is non-differentiable and noisy because the empirical win-rates are based on a finite number of games. These properties motivate the use of a black-box optimization method, discussed in the next section.

3.4 Evolution Strategies Training

The ES optimizer works on top of the feasibility loss (3.4). The parameter vector θ collects all weights and biases of the neural network controller; its dimension depends on the chosen architecture and is fixed once the architecture is specified.

Each ES iteration proceeds as follows.

1. **Population sampling.** A population of λ perturbations $\{\varepsilon_i\}_{i=1}^{\lambda}$ is sampled from a multivariate normal distribution $\mathcal{N}(0, \sigma^2 I)$. Candidate parameters are formed as $\theta_i = \theta + \varepsilon_i$.

2. **Loss evaluation.** For each θ_i, the simulator is called to estimate $\widehat{p}_s(\theta_i)$ on all scenarios $s \in S$, using a fixed number of games per scenario. The implementation runs these games in parallel across CPU cores, so that different individuals and scenarios can be evaluated concurrently. The feasibility loss $J_S(\theta_i)$ is then computed via (3.4).

3. **Weight construction.** The losses $J_S(\theta_i)$ are converted into weights w_i by ranking the individuals and assigning higher selection weights to lower-loss candidates. The weights are normalized so that $\sum_i w_i = 1$.

4. **Parameter update.** The mean parameter vector is updated by

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{\lambda} w_i \varepsilon_i, \tag{3.5}$$

   where $\alpha > 0$ is a learning rate specified in configuration.

The parameter update can be interpreted as a rank-based weighted recombination step that moves the parameter vector toward the best-performing perturbations.

Given the sampled candidates $\theta_i = \theta + \varepsilon_i$, with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2 I)$, and normalized weights $w_i \geq 0$, $\sum_{i=1}^{\lambda} w_i = 1$, assigned in decreasing order of performance (i.e., higher weights to lower-loss candidates), the update rule

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{\lambda} w_i \varepsilon_i$$

corresponds to a stochastic search step that shifts the mean parameter vector toward directions associated with better empirical performance. This interpretation is consistent with rank-based Evolution Strategies and weighted recombination mechanisms commonly used in ES and CMA-ES variants [6, 7, 8]. In the software, this logic is encapsulated in the `EvolutionStrategiesBBO` class in `es_bbo.py`, which is driven by scripts such as `run_es_once.py` and the QG orchestrator `qg_runner.py`.

The ES training phase terminates when either a maximum number of iterations is reached or when the empirical loss stops decreasing significantly. The final value of θ is then passed to the verification layer.

3.5 Sequential Scenario Verification

While ES uses relatively rough estimates of $\widehat{p}(\theta, s)$ (few games per scenario) to keep training affordable, the verification layer aims at providing accurate, scenario-wise information with explicit error control. For each $s \in S$, the implementation runs a sequential procedure that adaptively decides how many games to play.

Algorithm 5 mirrors the implementation in `sequential_verification.py`. Scenario-wise verification is carried out by repeatedly calling the match simulator and updating Wilson intervals until the stopping rule is met.

Wilson score intervals in practice

Let n_s be the number of games played so far under scenario s, and let k_s be the number of wins. The empirical win-rate under controller θ is

$$\widehat{p}(\theta, s) = \frac{k_s}{n_s}.$$

To quantify uncertainty, the implementation uses the Wilson score interval for binomial proportions [14]. Given a confidence level $1 - \delta$ and the corresponding normal quantile $z = z_{1-\delta/2}$, the Wilson interval is computed as

$$[\ell_s, u_s] = \frac{\widehat{p}(\theta, s) + \frac{z^2}{2n_s}}{1 + \frac{z^2}{n_s}} \pm \frac{z\sqrt{\frac{\widehat{p}(\theta,s)\left(1-\widehat{p}(\theta,s)\right)}{n_s} + \frac{z^2}{4n_s^2}}}{1 + \frac{z^2}{n_s}}. \tag{3.6}$$

In the code, this formula is implemented in a dedicated utility function, which takes the pair (k_s, n_s) and the confidence level as inputs and returns (ℓ_s, u_s).

Several alternative confidence intervals could have been used (for example, the Clopper–Pearson "exact" interval or Bayesian intervals based on a Jeffreys prior), but the Wilson interval offers a favorable compromise between accuracy and simplicity. It provides good coverage even for moderate sample sizes and probabilities close to 0 or 1, while remaining cheap to compute and balanced in its treatment of low and high win-rates. These properties make it well suited for the large number of binomial estimations required by the QG loop.

Sequential stopping rule

The sequential verifier for a single scenario operates in rounds:

1. Initialize $n_s = 0$, $k_s = 0$;
2. play a new batch of games under scenario s with the final controller θ, updating the counts (k_s, n_s);
3. recompute $\widehat{p}_s$ and the Wilson interval $[\ell_s, u_s]$;
4. check whether the interval radius

$$r_s = \frac{u_s - \ell_s}{2}$$

is below a prescribed accuracy ε, or whether a maximum number of games has been reached.

If the stopping condition is met, the scenario is classified using the following rules:

- *pass* if $\ell_s \geq \tau_{\text{verify}}$;
- *fail* if $u_s < \tau_{\text{verify}}$;
- *borderline* otherwise (by default treated as fail in the experiments).

The verifier returns, for each s, the final estimate $\widehat{p}_s$, the interval bounds, the number of games used, and the pass/fail decision. This information is later used both in the experimental chapter and in the counterexample-guided refinement of the scenario set.

3.6 Statistical Model Checking of Violation Probability

Scenario-wise verification controls the performance on the finite set S but does not, by itself, provide guarantees on the global violation probability $\gamma(\theta)$ in (3.2). In the implementation, we estimate instead the empirical violation probability $\gamma_{n_{\text{int}}}(\theta)$ defined in (3.3), where each scenario-level win-rate is approximated using a finite number n_{int} of simulated games. To estimate $\gamma_{n_{\text{int}}}(\theta)$ with absolute-error guarantees, the system runs a Statistical Model Checking (SMC) procedure [1, 2, 9, 15].

The SMC procedure samples scenarios $s^{(1)}, s^{(2)}, \ldots$ independently from the distribution D. For each sampled scenario $s^{(k)}$, a small internal experiment is performed:

1. a fixed number n_{int} of games is played under $s^{(k)}$ using the final controller θ;
2. an empirical win-rate $\widehat{p}_{n_{\text{int}}}(\theta, s^{(k)})$ is computed;
3. a violation indicator is defined as

$$Z_k = \begin{cases} 1, & \text{if } \widehat{p}_{n_{\text{int}}}(\theta, s^{(k)}) < \tau_{\text{verify}}, \\ 0, & \text{otherwise.} \end{cases}$$

The random variables Z_k are Bernoulli with mean $\gamma_{n_{\text{int}}}(\theta)$. After N sampled scenarios, the empirical violation probability is

$$\widehat{\gamma} = \frac{1}{N} \sum_{k=1}^{N} Z_k.$$

A Wilson score interval $[\ell_\gamma, u_\gamma]$ is constructed for $\gamma_{n_{\text{int}}}(\theta)$ at confidence level $1 - \delta$. The sampling process is run sequentially until either

$$\frac{u_\gamma - \ell_\gamma}{2} \leq \varepsilon_\gamma$$

or a maximum number of scenarios has been sampled.
The QG acceptance criterion is

$$u_\gamma \leq \varepsilon_{\text{viol}},$$

which supports, at confidence level $1 - \delta$, the statement that

$$\gamma_{n_{\text{int}}}(\theta) \leq \varepsilon_{\text{viol}}.$$

Thus, SMC provides statistical guarantees for $\gamma_{n_{\text{int}}}(\theta)$, which is used as a proxy for the true violation probability $\gamma(\theta)$. The approximation bias decreases as n_{int} increases, and under sufficiently large internal budgets the empirical quantity approaches the theoretical one.

The guarantees provided by the Wilson confidence interval are empirical and hold with high confidence under finite-sample estimation. Although the stopping rule is adaptive, the procedure is intended as a practical high-confidence certification mechanism rather than a formal sequential inference guarantee.

If the acceptance criterion is not satisfied, sampled scenarios with $Z_k = 1$ are stored as counterexamples and used to refine the training set.

The logic of Algorithm 6 is implemented in `smc.py`, which defines the SMC driver used by `qg_runner.py` during the verification phases of the QG loop.

3.7 Mapping between methods and Python modules

For clarity and reproducibility, we briefly summarize how the main methods described in this chapter are mapped to the Python codebase:

- **Match simulation** (Algorithm 1) is implemented by the chess engine and match driver functions in `chess_engine.py`, which expose routines to initialize positions, generate legal moves, apply moves, and evaluate outcomes.
- **Win-rate estimation** (Algorithm 2) is realized by helper functions in `batch_logging.py` and `sequential_verification.py`, which call the match simulator repeatedly and aggregate the resulting outcomes.
- **Feasibility loss computation** (Algorithm 3) and the ES update (Algorithm 4) are implemented in `es_bbo.py`, in the `EvolutionStrategiesBBO` class and related utilities.
- **Sequential scenario verification** (Algorithm 5) is implemented in `sequential_verification.py`. Scenario-wise verification is carried out by repeatedly calling the match simulator and updating Wilson intervals until the stopping rule is met.
- **SMC of violation probability** (Algorithm 6) is implemented in `smc.py`, which samples scenarios from the distribution configured in `config.py` and logs counterexamples.

- **QG loop orchestration** (Algorithm 7) is implemented in `qg_runner.py`, which coordinates ES training, scenario-wise verification, and SMC, and updates the scenario set with newly found counterexamples.

This explicit mapping highlights that the QG loop is not only specified at an abstract level but is fully realized in a concrete, reproducible software artifact.

3.8 Non-standard aspects of this QG instantiation

The chess case study considered in this thesis follows the general Quality-Guaranteed paradigm but combines several design choices that are not standard in reinforcement learning or in the game-playing literature:

- The ES optimizer is applied to a *feasibility loss* built around a win-rate requirement, rather than to a smooth reward function. This shifts the focus from maximizing average performance to satisfying an explicit constraint on scenario-wise win-rates.
- Statistical Model Checking is used to estimate the *violation probability* of this requirement under a scenario distribution, rather than to verify temporal-logic properties of a fixed model. The QG loop, therefore, reasons directly in terms of controller failure rates.
- Counterexample-guided refinement operates at the level of *scenarios*, adding seeds and opponent policies that have been observed to violate the requirement back into the training set. This creates a feedback loop in which verification actively shapes the optimization problem, in the spirit of CEGIS [11].
- The neural controller is deliberately compact and CPU-friendly. The goal is not to compete with state-of-the-art engines in playing strength, but to isolate and expose the behavior of the QG loop itself on a realistic yet tractable control problem.

Taken together, these choices lead to a QG instantiation that differs substantially from classical self-play pipelines and typical applications of SMC, and that is tailored to the goal of obtaining explicit, distribution-level guarantees on win-rate.

3.9 The Complete Quality-Guaranteed Loop

Putting everything together, the QG loop implemented in the project consists of the following steps.

1. **Initialization.** Choose an initial scenario set S_0, a scenario distribution D, training threshold τ_{train}, verification threshold τ_{verify}, absolute-error parameters ε and ε_γ, and a violation tolerance $\varepsilon_{\text{viol}}$. Initialize θ randomly and set $S \leftarrow S_0$.
2. **ES training.** Run the ES optimizer on $J_S(\theta)$ for a fixed number of iterations or until convergence, using the simulator to evaluate the loss.

3. **Scenario-wise verification.** For each $s \in S$, run the sequential verifier described in Section 3.5, obtaining $\widehat{p}_s$, a Wilson interval, and a pass/fail decision. Collect the set $F \subseteq S$ of non-passing scenarios (i.e., scenarios classified as FAIL or BORDERLINE).

4. **Global SMC.** Run the SMC procedure in Section 3.6 to estimate $\gamma(\theta)$, obtaining $\widehat{\gamma}$ and a Wilson interval $[\ell_\gamma, u_\gamma]$.

5. **Stopping test.** If $u_\gamma \leq \varepsilon_{\text{viol}}$, accept the controller: with probability at least $1 - \delta$, the violation probability is within tolerance. The loop terminates.

6. **Refinement.** Otherwise, collect the set C of scenarios sampled during SMC with $Z_k = 1$ and update $S \leftarrow S \cup C$. Optionally, adjust ES hyperparameters to give more weight to scenarios in $F \cup C$.

7. **Iteration.** Return to ES training with the enriched set S.

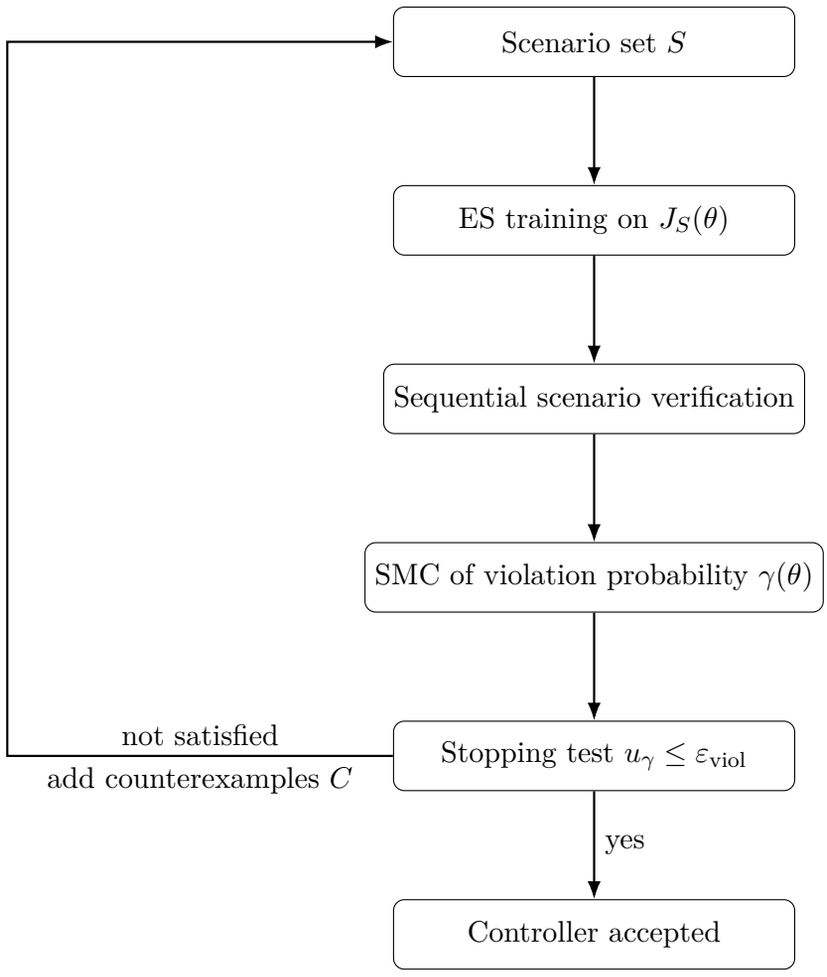

Figure 3.2. Logical Quality-Guaranteed learning loop for chess.

Although Figure 4.1 presents the modules in a pipeline-like layout, the interaction among components is not strictly unidirectional. Both the ES optimizer and the

verification modules repeatedly invoke the engine and the controller during match simulation. In this sense, the lower layers (engine and controller) act as callable services that are iteratively queried by higher-level components, rather than as passive stages in a single forward data flow.

In this way, the QG loop implemented in the project does not treat learning and verification as separate phases. Instead, they form a single iterative process: ES proposes controllers that reduce a requirement-centric loss, verification evaluates violation probability with explicit confidence bounds, and counterexamples are fed back into the next optimization cycle.

Algorithm 1 SimulateMatch(θ, s)

Require: controller parameters θ, scenario $s = (\sigma, o)$
Ensure: match outcome $Y \in \{0, 1\}$

```
init game state using seed σ
while game not terminal and ply < N_max do
    if designed player to move then
        choose move with controller f_θ (optionally ε-greedy)
    else
        choose move with opponent policy o
    end if
    apply move to the game state
end while
Y ← 1 if win else 0                          ▷ draw/loss count as failure
return Y
```

Algorithm 2 EstimateWinRate(θ, s, n)

Require: controller parameters θ, scenario $s = (\sigma, o)$, number of games n
Ensure: empirical win-rate $\widehat{p}(\theta, s)$

```
k ← 0                                                    ▷ win counter
for i ← 1 to n do
    σ_i ← DeriveSeed(σ, i)
    Y_i ← SimulateMatch(θ, (σ_i, o))
    k ← k + Y_i
end for
p̂(θ, s) ← k/n
return p̂(θ, s)
```

Algorithm 3 ComputeFeasibilityLoss$(\theta, S, \tau_{\text{train}}, n_{\text{games}})$

Require: controller parameters θ, training scenario set S, win-rate threshold τ_{train}, number of games per scenario n_{games}
Ensure: feasibility loss $J_S(\theta)$

$J_S(\theta) \leftarrow 0$
for all $s \in S$ **do**
  $\widehat{p}(\theta, s) \leftarrow$ EstimateWinRate$(\theta, s, n_{\text{games}})$
  $d_s \leftarrow \max\big(0,\ \tau_{\text{train}} - \widehat{p}(\theta, s)\big)$
  $J_S(\theta) \leftarrow J_S(\theta) + d_s$
end for
return $J_S(\theta)$

Algorithm 4 Evolution Strategies training on the feasibility loss $J_S(\theta)$

Require: initial parameters θ, scenario set S, population size λ, perturbation scale σ, learning rate α, number of games per scenario n_{games}
Ensure: updated parameters θ

for iteration $= 1, 2, \ldots$ until stopping criterion is met **do**
  for $i = 1$ to λ **do**
    sample $\varepsilon_i \sim \mathcal{N}(0, \sigma^2 I)$
    $\theta_i \leftarrow \theta + \varepsilon_i$
    for all $s \in S$ **do**
      $\widehat{p}(\theta_i, s) \leftarrow$ EstimateWinRate$(\theta_i, s, n_{\text{games}})$
    end for
    compute feasibility loss

$$J_S(\theta_i) = \sum_{s \in S} \max\big(0,\ \tau_{\text{train}} - \widehat{p}(\theta_i, s)\big)$$

  end for
  rank candidates $\{\theta_i\}$ by increasing loss $J_S(\theta_i)$
  assign normalized weights $\{w_i\}_{i=1}^{\lambda}$
  update parameters

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{\lambda} w_i \varepsilon_i$$

  if convergence criterion or maximum iterations reached **then**
    break
  end if
end for
return θ

Algorithm 5 Sequential scenario-wise verification using Wilson intervals

Require: controller θ, scenario set S, threshold τ_{verify}, accuracy ε, confidence level $1-\delta$, maximum games per scenario $n_{\max}$
Ensure: estimates $\{\widehat{p}(\theta, s)\}_{s \in S}$ and set of non-passing scenarios F

$F \leftarrow \emptyset$
for all $s \in S$ **do**
    $k_s \leftarrow 0$ ▷ wins
    $n_s \leftarrow 0$ ▷ games
    status$(s) \leftarrow$ UNDECIDED
end for
while there exists s with status$(s) =$ UNDECIDED **do**
    for all s with status$(s) =$ UNDECIDED **do**
        play a batch of games under (θ, s)
        update k_s and n_s
        $\widehat{p}(\theta, s) \leftarrow k_s / n_s$
        compute Wilson interval $[\ell_s, u_s]$ at level $1-\delta$
        $r_s \leftarrow (u_s - \ell_s)/2$
        if $r_s \le \varepsilon$ or $n_s \ge n_{\max}$ **then**
            if $\ell_s \ge \tau_{\text{verify}}$ **then**
                status$(s) \leftarrow$ PASS
            else if $u_s < \tau_{\text{verify}}$ **then**
                status$(s) \leftarrow$ FAIL
                $F \leftarrow F \cup \{s\}$
            else
                status$(s) \leftarrow$ BORDERLINE
                $F \leftarrow F \cup \{s\}$
            end if
        end if
    end for
end while
return $\{\widehat{p}(\theta, s)\}_{s \in S}, F$

Algorithm 6 SMCVIOLATION$(\theta, D, \tau_{\text{verify}}, n_{\text{int}}, \varepsilon_\gamma, \delta, N_{\max})$

Require: controller parameters θ, scenario distribution D, verification threshold τ_{verify}, number of internal games per scenario n_{int}, accuracy ε_γ, confidence level $1-\delta$, maximum number of sampled scenarios $N_{\max}$

Ensure: empirical violation estimate $\widehat{\gamma}$, Wilson interval $[\ell_\gamma, u_\gamma]$ for $\gamma_{n_{\text{int}}}(\theta)$, and counterexample set C

$C \leftarrow \emptyset$
$N \leftarrow 0$ ▷ number of sampled scenarios
$K \leftarrow 0$ ▷ number of violations
repeat
  sample scenario $s \sim D$
  $\widehat{p}_{n_{\text{int}}}(\theta, s) \leftarrow$ ESTIMATEWINRATE$(\theta, s, n_{\text{int}})$
  if $\widehat{p}_{n_{\text{int}}}(\theta, s) < \tau_{\text{verify}}$ **then**
    $K \leftarrow K + 1$
    $C \leftarrow C \cup \{s\}$
  end if
  $N \leftarrow N + 1$
  $\widehat{\gamma} \leftarrow K/N$
  compute Wilson interval $[\ell_\gamma, u_\gamma]$ for the Bernoulli violation probability based on (K, N)
  $r_\gamma \leftarrow (u_\gamma - \ell_\gamma)/2$
until $r_\gamma \le \varepsilon_\gamma$ **or** $N \ge N_{\max}$
return $(\gamma_b,\ [\ell_\gamma, u_\gamma],\ C)$

Algorithm 7 Quality-Guaranteed (QG) learning loop for chess

Require: initial scenario set S_0, scenario distribution D, training threshold τ_{train}, verification threshold τ_{verify}, number of games per scenario n_{games}, number of internal games per scenario n_{int}, local accuracy ε, global accuracy ε_γ, confidence level $1-\delta$, maximum number of sampled scenarios $N_{\max}$, violation tolerance $\varepsilon_{\text{viol}}$

Ensure: controller parameters θ

$S \leftarrow S_0$
initialize θ randomly
while true **do**
  $\theta \leftarrow$ ESTRAIN$(\theta, S, \tau_{\text{train}}, n_{\text{games}})$
  $(\{\widehat{p}_s\}_{s\in S}, F) \leftarrow$ SEQUENTIALVERIFY$(\theta, S, \tau_{\text{verify}}, \varepsilon, \delta, n_{\max})$
  $(\gamma_b,\ [\ell_\gamma, u_\gamma],\ C) \leftarrow$ SMCVIOLATION$(\theta, D, \tau_{\text{verify}}, n_{\text{int}}, \varepsilon_\gamma, \delta, N_{\max})$
  if $u_\gamma \le \varepsilon_{\text{viol}}$ **then**
    return θ
  else
    $S \leftarrow S \cup C$
  end if
end while

Chapter 4

Implementation

This chapter explains how the methods from Chapter 3 are realized in the actual Python project. The goal is not to restate the theory, but to show in a concrete and reproducible way how the abstract objects of the Quality-Guaranteed (QG) framework —scenarios, win-rates, feasibility loss, Evolution Strategies (ES), Wilson intervals, and Statistical Model Checking (SMC)—are turned into modules, functions, and data structures, following the standard treatments in ES [6, 7, 8], Wilson intervals [14], and SMC [1, 2, 9, 15].

All the components described here are part of the public codebase of the project. The core elements of the QG loop are implemented in `chess_engine.py` (chess engine and match simulation), `nn_player.py` and `nn_core.py` (controller and neural network), `es_bbo.py` (ES optimizer), `sequential_verification.py` (scenario-wise checks), `smc.py` (global SMC), and `qg_runner.py` (QG driver and orchestration). The implementation follows closely the algorithms in Chapter 3.

4.1 Software Architecture Overview

The implementation is organized as a thin stack of Python modules and packages, each mirroring one logical layer of the QG pipeline introduced in Chapter 3. A schematic view is shown in Figure 4.1.

In the reference implementation these layers are realized as Python packages (for example, `engine`, `controller`, `optim`, `verify`, `scenarios`, `utils`). The precise directory layout is conventional and can be changed without touching the underlying logic, as long as the same interfaces are preserved.

At runtime, the data flow can be summarized as follows:

1. the scenario manager constructs or selects a scenario s and hands it either to the ES layer (during optimization) or to the verification layer (during checks);
2. the engine initializes the corresponding chess position, returning a `GameState` object representing the current board;
3. the controller encodes the state into a vector, calls the neural network to obtain a score, and chooses a move according to an ε-greedy policy;

4. the engine applies the move, queries the opponent policy, and advances the state until a terminal condition or a maximum-plies cap is reached;

5. match outcomes are passed upstream and aggregated into win-rates, feasibility losses, and violation indicators by the ES and verification modules.

Each step instantiates, in a one-to-one fashion, the corresponding mathematical object in Chapter 3: a single match realizes the Bernoulli trial SimulateMatch of Algorithm 1, empirical win-rates are computed as in Algorithm 2, the ES module minimizes $J_S(\theta)$ in Equation (3.4), and the verification modules implement the sequential and SMC procedures of Algorithms 5 and 6.

4.2 Engineering Choices

The architecture was intentionally designed to be simple, transparent, and reproducible, rather than competitive with production chess engines. Several design decisions were guided by this constraint and by the need to keep the QG loop executable on a standard laptop.

Execution environment. All experiments are run on a standard multi-core CPU, using Python 3 and only lightweight numerical libraries. The implementation does not require a GPU. This matches the design goal in Chapter 3: the main bottleneck is the number of simulated games required for ES and SMC, not the complexity of individual forward passes.

Custom engine rather than external libraries. Instead of delegating the game logic to an existing package such as `python-chess` [4], the project uses a compact in-house engine embedded in `chess_engine.py`. This eliminates an external dependency and ensures that the exact transition function used in experiments is fully visible and modifiable. It also removes ambiguity on how special rules (castling, en passant, repetitions) are handled: the behavior used in the QG analysis is exactly the behavior implemented in the simulator.

CPU-only controller. The neural controller is deliberately shallow and compact, with a fully-connected architecture implemented in `nn_core.py`. The controller can be evaluated millions of times in a single run without requiring specialized hardware. From the perspective of QG, this is preferable to a deeper model: the bottleneck is the number of games required for ES and SMC, not the expressive power of the network.

ES instead of gradient-based RL. Gradient-based reinforcement learning would require a more complex training loop, explicit value or policy targets, and several stabilization tricks. In this project the target quantity is the non-differentiable feasibility loss $J_S(\theta)$, built from empirical win-rates and positive-part operations as in Equation (3.4). ES, implemented in `es_bbo.py`, treats the whole simulator—engine plus opponents plus controller—as a black box and only needs scalar losses.

The same optimizer would work unchanged if the engine or the controller were replaced, following standard ES formulations [6, 7, 8].

Scenario-centered design. Finally, the decision to expose scenarios as first-class objects is a direct reflection of the QG paradigm formalized in Section 3.2. The scenario manager keeps track of the current set S, the initial set S_0, and the counterexample buffer C. ES never trains on an implicit "stream of games"; it always operates on an explicit finite set of scenarios that grows over iterations, which makes the evolution of the QG loop easier to inspect and debug.

4.3 Chess Engine Core

The chess engine is responsible for representing game states and advancing them according to the rules of chess. In the implementation, a position is represented by an instance of the `GameState` class in `chess_engine.py`. The class maintains:

- the occupancy of each of the 8×8 squares (piece type or empty);
- whose turn it is to move;
- castling rights for both sides;
- an optional *en passant* target file;
- half-move and full-move counters.

This representation matches the abstract notion of position in Section 2.2. The engine exposes, through the public interface of `GameState`, the two basic operations used throughout the project:

- `generate_legal_moves()`, which returns all legal moves in the current state;
- `make_move(move)`, which applies a move and returns the successor state.

The internal logic follows the two-step strategy described in Section 2.2. First, for each piece belonging to the side to move, the engine constructs a list of *pseudo-legal* moves: moves that respect the movement rules of the piece but ignore king safety. Then each candidate move is tested by applying it on a temporary copy of the position and checking whether the moving side's king is left in check. Moves that fail this test are discarded.

Special moves are handled explicitly:

- *Castling* is permitted only if neither king nor rook has moved, the intermediate squares are empty, and none of the squares that the king passes through is attacked.
- *En passant* captures are allowed only in the move immediately following a pawn double step of the opposing side, and only from eligible neighboring pawn squares.

- *Promotion* moves carry a promotion piece type; the engine automatically generates one legal move per admissible promotion piece.

Draw conditions (stalemate, dead positions due to insufficient material, repetitions and the fifty-move rule) are checked at the end of each move. The engine keeps track of the necessary information so that long games eventually terminate even in pathological positions. In addition, the QG loop imposes a global maximum on the number of plies per game; if the cap is reached, the match is treated as non-winning for the controller, consistent with the must-win requirement used to define the Bernoulli outcomes in Section 2.5.

A small suite of unit tests accompanies `GameState`. These tests cover basic movement of each piece, detection of illegal self-check moves, castling rules, en passant edge cases, and draws by repetition. They act as a regression suite and give practical confidence that the simulator used by ES and SMC respects the rules assumed in Chapter 2 and Chapter 3.

4.4 State Encoding and Neural Network

The controller layer links the symbolic game state maintained by the engine with the numerical vector expected by the neural network. In the implementation, this mapping is provided by utility functions in `nn_player.py` and `nn_core.py`, instantiating the encoding pipeline of Section 2.3.

Given a `GameState` object, the encoder constructs a vector $x \in \mathbb{R}^d$ according to the following pattern:

1. **Piece planes.** For each piece type and color, an 8×8 binary plane is created. The entry for a square is 1 if the square contains that piece type and 0 otherwise.

2. **Global features.** A small set of scalar features encodes the side to move, castling rights and basic move-count information.

3. **Flattening.** All planes and scalar features are concatenated and flattened into a vector x of fixed dimension `D_INPUT`, defined in `nn_player.py`.

The encoder is deterministic: the same `GameState` always produces the same vector. This determinism is crucial when scenarios are replayed with the same seed during debugging or verification.

The neural network itself is a fully connected feed-forward model $f_\theta : \mathbb{R}^d \to \mathbb{R}$, implemented in `nn_core.py`. The model consists of an input layer of size `D_INPUT`, one or more hidden layers with non-linear activation functions, and a final scalar output. All weights and biases are stored in a single parameter vector θ, which is the object manipulated by ES. Access to θ is encapsulated in a thin wrapper class `NNPlayer` in `nn_player.py`.

Move selection works as follows, instantiating the policy described in Section 2.3 and Algorithm 1:

1. the controller calls `generate_legal_moves()` on the current `GameState`;

2. each move is applied virtually to obtain a successor state;
3. the successor state is encoded to a vector x and scored by the network as $f_\theta(x)$;
4. the move with the highest score is selected, possibly replaced with a random move with probability ε (an ε-greedy exploration mechanism [12]).

This procedure instantiates the abstract controller discussed in Chapter 3 as a concrete Python object that can be dropped into the simulator or into an interactive GUI.

4.5 Evolution Strategies Module

The ES optimizer is implemented in `es_bbo.py` and built around the feasibility loss $J_S(\theta)$ defined in Equation (3.4). The module exposes a class `EvolutionStrategiesBBO` that maintains:

- the current parameter vector θ;
- ES hyperparameters: population size λ, perturbation scale σ, learning rate α;
- the current training scenario set S;
- configuration for the number of games per scenario and per individual.

Each ES iteration follows the steps of Algorithm 4:

1. sample perturbations ε_i and build candidates $\theta_i = \theta + \varepsilon_i$;
2. for each θ_i, call the simulator to estimate empirical win-rates $\widehat{p}_s(\theta_i)$ on all scenarios $s \in S$;
3. compute the feasibility loss $J_S(\theta_i)$ from these win-rates;
4. rank candidates by loss and convert ranks into normalized weights w_i;
5. update θ according to the ES rule in Equation (3.5).

Game evaluations are parallelized using a pool of worker processes. Each worker is assigned a batch of candidate parameters and scenarios, which it evaluates by repeatedly calling the match-simulation routines in `chess_engine.py`. Aggregated statistics (wins, losses, game counts) are sent back to the main process, which computes the feasibility loss and updates θ.

The ES module logs, for each iteration, the value of $J_S(\theta)$, the average win-rate over the current training set S, and basic timing information. These logs are later reused in the experimental chapter to plot learning curves and to diagnose convergence issues.

4.6 Sequential Verification Module

Scenario-wise verification is implemented in `sequential_verification.py`. The module provides a function that takes as input the final controller parameters θ, the scenario set S, the threshold τ, accuracy and confidence parameters, and returns a detailed report for each scenario.

Internally, for each $s \in S$, the verifier maintains:

- win and game counters (k_s, n_s);
- the current empirical win-rate $\widehat{p}_s$;
- the current Wilson interval $[\ell_s, u_s]$ [14];
- a status flag (undecided, pass, fail, or borderline).

Verification proceeds in rounds, following Algorithm 5:

1. identify all scenarios whose status is still undecided;
2. for each such scenario, play a new batch of games under the controller θ, updating k_s and n_s;
3. recompute $\widehat{p}_s$ and the Wilson interval $[\ell_s, u_s]$;
4. if the interval radius is below the target accuracy ε, or if a maximum game budget has been exhausted, classify the scenario as pass, fail, or borderline according to the rules in Section 3.5.

The Wilson interval computation [14] is factored out in a small utility function, reused by the SMC engine to avoid duplicated code. At the end of the run, the verifier writes a machine-readable summary (for example, a CSV file) containing one row per scenario with the final win-rate, confidence interval, status and number of games. This file serves both as documentation of the verification phase and as an input to the counterexample management logic.

4.7 Statistical Model Checking Engine

The SMC component is implemented in `smc.py`. Its role is to estimate the empirical violation probability $\gamma_{n_{\text{int}}}(\theta)$, which serves as a Monte Carlo approximation of the true violation probability $\gamma(\theta)$ defined in Equation (3.2), together with an absolute-error guarantee, as formalised in Section 3.6 and in the SMC literature [1, 2, 9, 15].

The SMC engine takes as arguments:

- the scenario distribution D, implemented as a joint sampler over seeds and opponent identifiers;
- the final controller parameters θ;
- the number of internal games per sampled scenario;

- accuracy and confidence parameters ε_γ and δ.

The engine then executes Algorithm 6: it repeatedly samples scenarios $s^{(k)} \sim D$, plays the prescribed number of games under each, estimates a scenario-level win-rate, and converts it into a violation indicator Z_k. After each new scenario, it updates the empirical violation probability $\widehat{\gamma}$ and the Wilson interval $[\ell_\gamma, u_\gamma]$. The process stops when the interval radius falls below ε_γ or when a pre-specified maximum number of scenarios has been reached.

All scenarios with $Z_k = 1$ are saved in a counterexample list together with the estimated win-rate. The function returns the final estimate $\widehat{\gamma}$, the corresponding Wilson interval, the number of sampled scenarios, and the list of counterexamples. These outputs are consumed by `qg_runner.py` to decide whether to stop the QG loop or to enrich the training set.

4.8 Scenario and Counterexample Management

Scenarios are represented as lightweight Python objects containing:

- a 64-bit integer seed σ;
- an opponent identifier (for example, *random* or *greedy-material-with-noise*);
- optional metadata such as the last estimated win-rate or a tag marking the scenario as counterexample.

This representation instantiates the abstract notion of scenario $s = (\sigma, o)$ introduced in Section 3.2. The scenario manager, implemented as a small module in the `scenarios` package, provides utilities to:

- construct an initial training set S_0 by combining a fixed list of opponents with seeds sampled from a given range;
- serialize and deserialize scenario sets to and from disk (for example, in JSON format);
- merge one or more counterexample lists into the current set S, avoiding duplicates;
- optionally prioritize scenarios in S by assigning them weights based on past performance during ES training.

At the end of each QG iteration, `qg_runner.py` collects the failing scenarios from the sequential verifier and the violating scenarios from SMC, merges them into a buffer C, and updates the training set to $S \leftarrow S \cup C$, as in Section 1 and in counterexample-guided synthesis approaches [11]. Subsequent ES runs therefore operate on a richer set of situations, and the feasibility loss becomes more demanding over time.

4.9 Logging, Reproducibility, and Tools

Reproducibility is a key part of the project: once a QG run has been executed, it should be possible to re-run it and obtain the same sequence of matches and the same statistics. To this end, the implementation includes a simple but explicit logging and configuration layer.

At the level of individual matches, the logging utilities (implemented in the files `match_logging.py` and `batch_logging.py`) record:

- the scenario identifier and seed;
- a compact representation of the move sequence;
- the final outcome and game length.

At the level of complete runs (ES, sequential verification, and SMC), the system logs:

- the full configuration used (ES hyperparameters, QG thresholds, scenario sets);
- loss trajectories and aggregated win-rates versus iteration number;
- SMC estimates of $\gamma(\theta)$ and the associated confidence intervals;
- high-level timing information for each phase of the QG loop.

Configuration parameters are centralized in a module `config.py`, which is imported by all other modules. Random seeds are drawn from this configuration and propagated down to the engine, the controller, ES and verification code, so that the pseudo-random behavior of the entire system can be controlled from a single place. In practice, this means that rerunning `qg_runner.py` with the same configuration file reproduces the same sequence of games, up to floating-point differences.

On top of the core components, a lightweight graphical interface built on the same engine and controller allows one to inspect logged games and to play interactively against the trained controller. While not essential to the QG methodology itself, this interface is useful for sanity checks: it offers a human-readable view of the learned behavior and helps confirm that the theoretical guarantees are attached to a controller that plays legal and recognizably “chess-like” games.

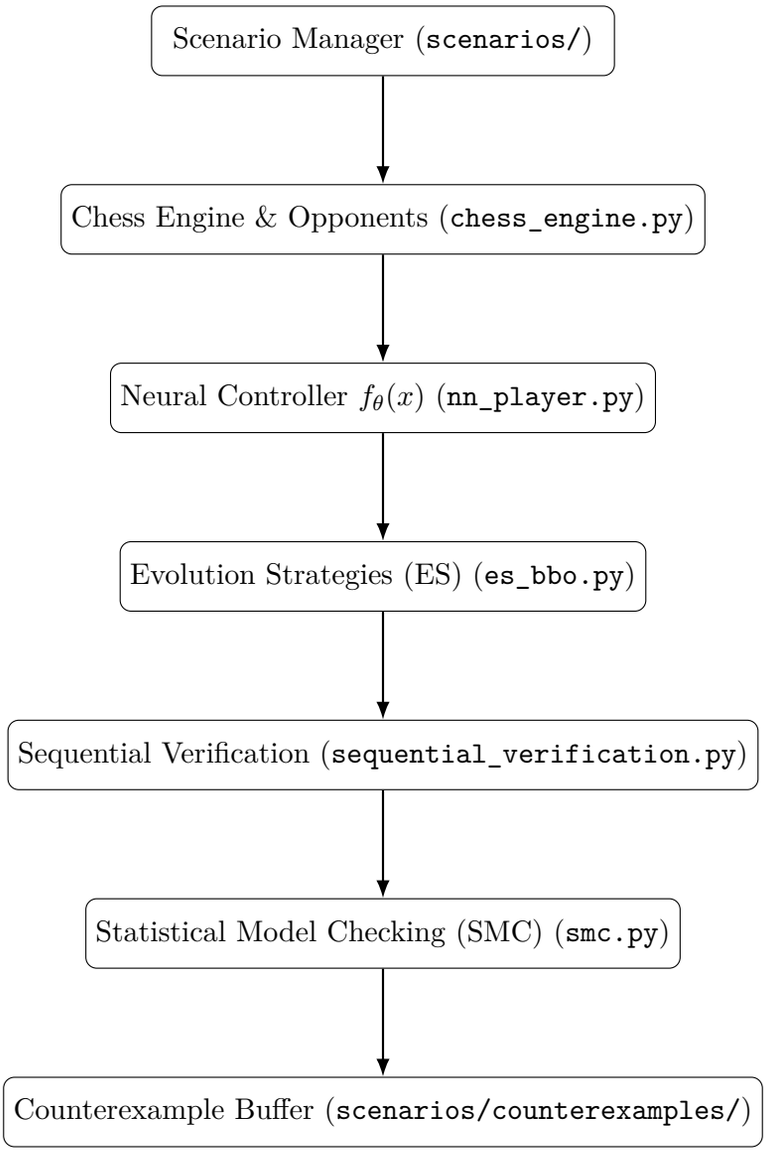

Figure 4.1. Software architecture of the Quality-Guaranteed chess system. Rectangles indicate Python modules or packages; arrows indicate the main direction of data flow.

Chapter 5

Experimental Results

This chapter reports the empirical behavior of the chess case study under the Quality-Guaranteed (QG) learning loop described in Chapter 3 and implemented as in Chapter 4. The goals are:

- to document how Evolution Strategies (ES), sequential verification and Statistical Model Checking (SMC) interact in practice on the chess domain;
- to quantify, with explicit confidence bounds, the win-rate and violation probability achieved by the neural controller with respect to the requirement of Section 3.2;
- to account for the overall simulation budget spent by each component of the QG loop.

Rather than focusing on individual code-level details, this chapter treats the implementation as a complete experimental pipeline and analyzes: training behavior under ES, scenario-wise verification results, SMC estimates of violation probability, match-level statistics against a set of baseline opponents, and the distribution of simulation costs across runs and phases.

5.1 Experimental Setup

All experiments follow the formal framework of Chapter 3. We briefly summarize here the concrete choices that are relevant for the interpretation of the plots and tables.

5.1.1 Controller and opponents

In all runs, the controller is the feed-forward neural network $f_\theta(x)$ introduced in Section 4.4, wrapped in the `NNPlayer` class. The network architecture and input encoding are fixed throughout the experiments; only the parameter vector θ is optimized.

The opponent library mirrors the policies used when defining scenarios in Section 3.2. In particular, the experiments use:

- a *random* opponent, which selects a uniformly random legal move in every position;
- a *greedy-material* opponent, which evaluates moves according to a material heuristic and chooses the move that maximizes immediate material gain;
- a *PSQT-greedy* opponent, which combines a piece-square-table evaluation with a greedy selection rule;
- a depth-limited minimax opponent, which evaluates a small tree of continuations with a fixed-depth minimax search and a simple evaluation function [5].

These opponents are not meant to approximate modern engines; they are simple, transparent baselines that stress different aspects of the controller.

5.1.2 Scenarios and requirements

Scenarios are defined exactly as in Section 3.2: each scenario $s = (\sigma, o)$ fixes a pseudo-random seed σ and an opponent policy o. The seed initializes all pseudo-random generators (controller exploration, opponent noise, and tie-breaking in move selection), making each scenario fully reproducible.

The experiments distinguish between:

- a finite *training set* S, used in ES and in sequential scenario-wise verification;
- a scenario distribution D, which samples seeds and opponent types and is used by SMC to estimate the violation probability $\gamma(\theta)$ in (3.2).

To distinguish between optimization guidance and final performance guarantees, two different success thresholds are employed in this study. During ES training, the feasibility loss is defined with respect to a threshold $\tau_{\text{train}} = 0.5$, enforcing a must-win constraint at the level of individual scenarios in S. Sequential scenario-wise verification then re-estimates the per-scenario win-rates at a stricter threshold $\tau_{\text{verify}} = 0.6$, while SMC evaluates the global violation probability $\gamma(\theta)$ under D. As in the rest of the thesis, draws are treated as failures, so the requirement is genuinely a *must-win* constraint. In the abstract framework of Chapter 3, typical thresholds lie in the range 0.7–0.9; for this concrete case study, the milder choices $\tau = 0.5$ and $\tau = 0.6$ strike a pragmatic balance between a non-trivial requirement and a feasible simulation budget.

Scenario distribution D (used in SMC). In SMC, each sampled scenario $s = (\sigma, o)$ is generated by drawing: (i) a seed σ from the configured seed range (uniformly at random), and (ii) an opponent type o from the opponent library according to fixed sampling weights specified in the run configuration/logs. Unless stated otherwise, games start from the standard initial chess position.

Definition check (outcome encoding consistency). Throughout this chapter, a game outcome is encoded as a Bernoulli variable $Y(\theta, s) \in \{0, 1\}$, where $Y = 1$ denotes a *win* for the controller and $Y = 0$ denotes *draw or loss*. This must-win encoding is used consistently across ES training, sequential verification, and SMC.

Important note on cross-phase comparability. Even with a consistent outcome encoding, win-rate estimates across phases are not directly comparable because they are produced under different experimental regimes (different sample sizes and, depending on the run configuration, potentially different settings such as exploration rate ε, plies caps, or opponent/seed sources). For this reason, the ES plots should be read as a *noisy optimization proxy*, while sequential verification and SMC provide the statistically controlled estimates reported later in the chapter.

5.1.3 Simulation budget and stopping criteria

For each combination of controller parameters, scenario and experiment type, only a finite number of games can be simulated. The numerical results reported below are therefore always based on *estimates*:

- during ES training, the feasibility loss $J_S(\theta)$ is evaluated using a fixed number of games per scenario, trading accuracy for speed;
- during scenario-wise verification, the number of games per scenario is chosen adaptively by the sequential Wilson-interval procedure of Section 3.5, up to a maximum cap (50 games in runs QualityA/B and 75 games in runs QualityC/D);
- during SMC, the number of scenarios sampled from D grows sequentially until the Wilson interval for $\gamma(\theta)$ reaches the target accuracy or a maximum number of scenarios is reached.

In all cases, the stopping rules are driven by the absolute-error guarantees introduced in Chapter 3: additional games or scenarios are simulated only as long as they meaningfully reduce statistical uncertainty.

The experiments reported in this chapter are based on four QG runs (reported as `logs_runQualityA–D`), along with a richer ES-only run (`logs_runRich2`) used to stress-test the optimization component. The resulting simulation budget is summarized in Table 5.3 in Section 5.5.

5.2 ES Training behavior

On comparability across phases. The reported win-rates are computed under different sampling regimes: ES uses small fixed batches for speed, sequential verification uses larger scenario-wise budgets with confidence intervals, and SMC evaluates scenarios drawn from D with a fixed internal budget n_{int}. For this reason, values across these blocks should be interpreted within their phase-specific context rather than compared one-to-one. We first analyze how the ES optimizer behaves when minimizing the feasibility loss $J_S(\theta)$ in (3.4). The key quantities of interest are:

- the per-scenario empirical win-rates achieved at the end of ES;
- the variability of training performance across seeds and across runs;
- aggregate statistics such as median and minimum win-rate.

Figure 5.1 visualizes the final ES win-rate for each scenario and quality run, while Figure 5.2 aggregates the distributions across runs.

Remark on evaluation protocols. During Evolution Strategies (ES) training, candidate policies are evaluated using a small, fixed number of games per scenario, providing a fast but high-variance estimate of performance that is sufficient for guiding the search. Sequential scenario-wise verification, on the other hand, allocates games adaptively for each scenario and relies on Wilson confidence intervals to achieve the prescribed accuracy and confidence requirements. Therefore, ES training curves and scenario-wise verification results refer to different estimation protocols and are not directly comparable in terms of absolute win-rate values.

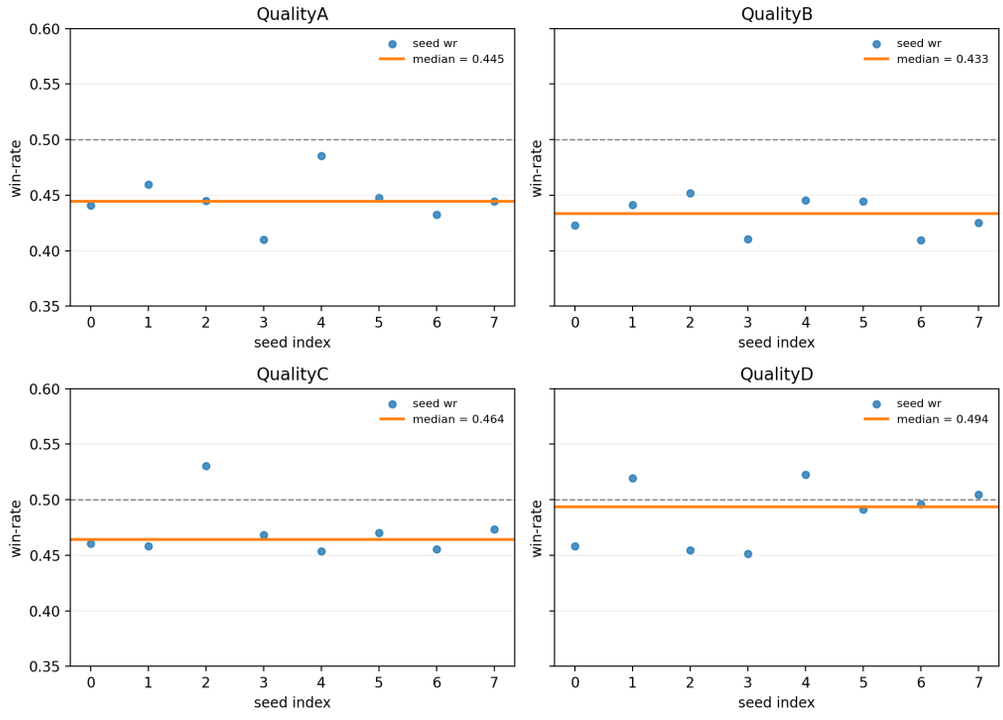

Figure 5.1. Final ES win-rate on each of the eight training scenarios for runs QualityA–D. Each point corresponds to a scenario; the orange line marks the median across scenarios within a run, and the dashed horizontal line marks the training threshold $\tau = 0.5$.

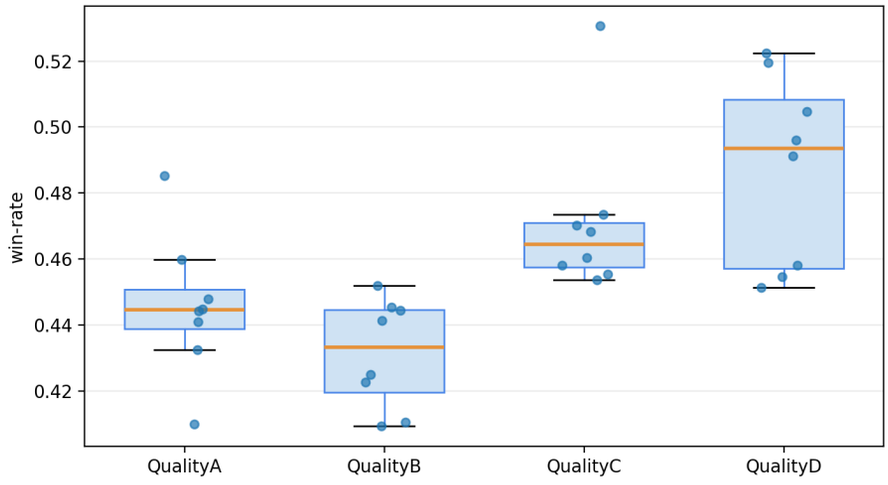

Figure 5.2. Aggregate distribution of ES win-rates for runs QualityA–D, shown as boxplots. For each run, the box spans the first and third quartiles; the orange segment is the median; whiskers extend according to the boxplot convention, and overlaid points show individual scenarios.

5.2.1 Training win-rates on the scenario set

For a given controller θ and scenario s, the empirical training win-rate is defined as

$$\widehat{p}(\theta, s) \;=\; \frac{\#\text{ wins on } s}{\#\text{ games played on } s},$$

computed from a small batch of training games. The overall training performance can then be summarized by the median scenario win-rate $\tilde{p}$, by the worst-case scenario win-rate $p_{\min}$, and by the fraction of scenarios that appear to satisfy the requirement at the training resolution.

Across the four QG runs (`logs_runQualityA–D`) with $|S| = 8$ scenarios and training threshold $\tau = 0.5$, a consistent pattern emerges. All runs end with median scenario win-rates in the range $0.433 \leq \tilde{p} \leq 0.494$, i.e. close to but still slightly below the threshold $\tau = 0.5$. The worst-case scenario win-rate remains between 0.409 and 0.454, so no run manages to satisfy the must-win requirement on *all* scenarios. It is important to note that these statistics summarize the overall distribution of empirical win-rates across the scenario set and should not be interpreted as threshold-based pass/fail indicators. In particular, although the median and worst-case win-rates lie below the target threshold $\tau = 0.5$, a non-negligible fraction of scenarios still attains empirical win-rates above τ, as discussed below. However, the fraction of scenarios whose training estimate exceeds τ increases across runs: it is 0% for runs A and B, 12.5% for run C and 37.5% for run D. This confirms that ES systematically improves the controller, but also highlights the limitations of the current network architecture and simulation budget: even after optimization, the controller remains only marginally below the desired specification. These trends are visible at a glance in Figures 5.1 and 5.2.

5.3 Scenario-wise Verification Results

After ES training, the controller is passed to the sequential verifier of Section 3.5. The goal is to obtain, for each scenario $s \in S$:

- an empirical win-rate $\widehat{p}_s$ computed with a larger number of games than in ES training;
- a Wilson confidence interval $[\ell_s, u_s]$ with prescribed coverage;
- a pass/fail decision relative to the must-win requirement at threshold $\tau = 0.6$.

The verifier stores its output in a machine-readable table containing one row per scenario, including the total number of games, the final win-rate estimate, the Wilson interval and the pass/borderline/fail classification. For the four QG runs considered here, the resulting statistics are summarized in Table 5.1.

Run	N	Pass	Borderline	Fail	$p_{\min}$	$p_{\max}$
`logs_runQualityA`	8	0.0%	100.0%	0.0%	0.680	0.700
`logs_runQualityB`	8	0.0%	100.0%	0.0%	0.680	0.700
`logs_runQualityC`	8	0.0%	100.0%	0.0%	0.587	0.640
`logs_runQualityD`	8	0.0%	100.0%	0.0%	0.600	0.653

Table 5.1. Sequential verification summary across QG runs (threshold $\tau = 0.6$). For each run, N is the number of scenarios in S, while $p_{\min}$ and $p_{\max}$ denote the minimum and maximum empirical win-rates observed at the end of verification.

Figure 5.3 shows how many verification games are played on each scenario across runs, while Figure 5.4 displays the final win-rates and Wilson intervals for each run and scenario, color-coded by difficulty.

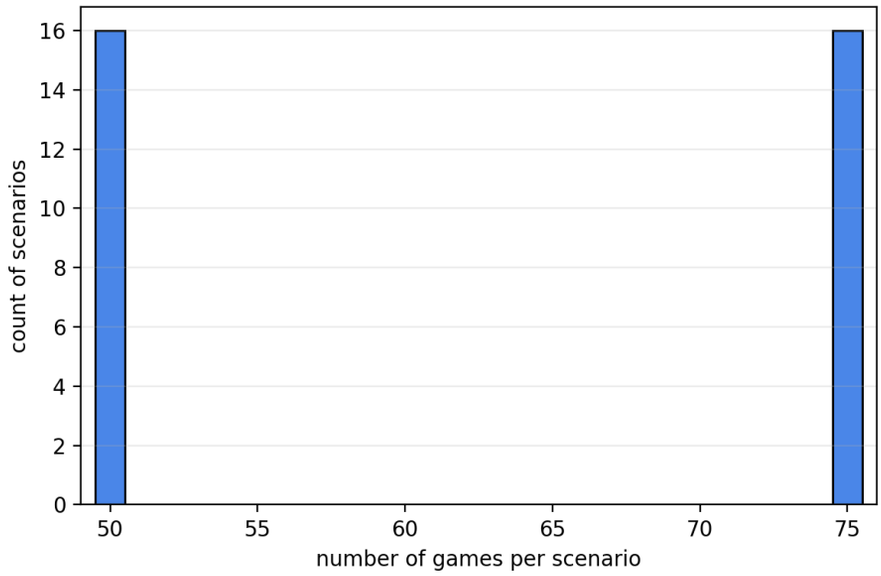

Figure 5.3. Distribution of the total number of verification games played per scenario across runs QualityA–D. All scenarios reach the maximum allowed budget (50 or 75 games depending on the run), reflecting the fact that all of them are statistically ambiguous at threshold $\tau = 0.6$.

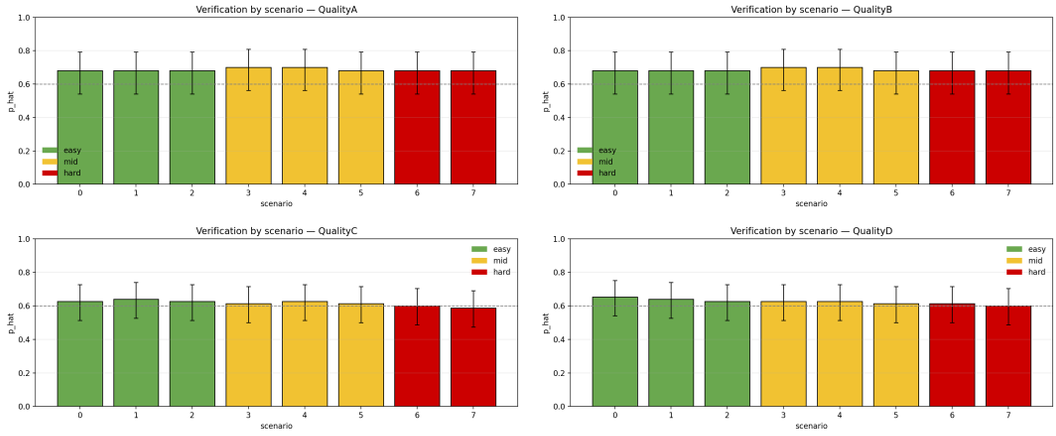

Figure 5.4. Scenario-wise verification results for runs QualityA–D. Bars show the empirical win-rates $\widehat{p}_s$ for each scenario, vertical segments denote Wilson confidence intervals, and colors encode scenario difficulty (easy, mid, hard). The dashed horizontal line marks the verification threshold $\tau = 0.6$.

5.3.1 Sample sizes and interval widths

The sequential procedure adaptively decides how many games to play on each scenario. In runs QualityA and QualityB the maximum number of games per scenario is capped at 50; in runs QualityC and QualityD this cap is raised to 75. As shown in Figure 5.3, every scenario in every run reaches the corresponding cap, so the final Wilson intervals are as tight as the budget allows.

Despite the fixed caps, the resulting Wilson intervals are reasonably narrow. For example, in runs QualityA and B the estimated win-rates lie in the range $[0.68, 0.70]$, well above τ_{verify} at the level of point estimates, although not yet conclusively separated from τ_{verify} at the chosen confidence level. In runs QualityC and D the range widens to $[0.587, 0.653]$: some scenarios lie very close to the threshold (with $\widehat{p}_s \approx 0.60$), while others are more clearly above it. This pattern is visible in Figure 5.4, where the harder (red) scenarios tend to cluster around the dashed threshold line.

5.3.2 Pass/borderline/fail decisions

Each scenario is ultimately classified into one of three categories (Algorithm 5):

- *pass* if the lower bound ℓ_s is safely above τ;
- *fail* if the upper bound u_s is clearly below τ;
- *borderline* if the final interval straddles τ, in which case the scenario is conservatively treated as failing from the QG perspective.

In the present chess case study, all 8 scenarios in each of the four QG runs end up in the *borderline* category: there are no scenarios whose lower bound lies safely above $\tau = 0.6$, and no scenarios whose upper bound lies safely below it. This is consistent with the values in Table 5.1 and with Figure 5.4, where all error bars intersect the dashed line. The verifier therefore reports empirical win-rate estimates around 60%–70% on the training scenarios together with Wilson intervals. However, the available budget is not sufficient to conclude pass/fail at $\tau = 0.6$, and all scenarios are conservatively treated as potential violations in the QG loop.

From a statistical point of view, this outcome is still informative. It quantifies precisely how close the controller is to the specification, and it identifies the most critical scenarios (those with $\widehat{p}_s$ just above or below τ) as natural candidates for further QG iterations or for architectural changes in the controller.

5.4 SMC Estimates of Violation Probability

Scenario-wise verification controls performance on the finite set S, but does not by itself bound the violation probability $\gamma(\theta)$ over the broader scenario distribution D. For this, the SMC procedure of Section 3.6 is invoked.

5.4.1 Violation indicators and Wilson intervals

For each sampled scenario $s^{(k)} \sim D$, SMC runs a small internal experiment: a fixed number of games is played under the final controller, and the empirical win-rate $\widehat{p}_{n_{\text{int}}}(\theta, s^{(k)})$ is compared to τ_{verify}. This yields a violation indicator $Z_k \in \{0, 1\}$, which is treated as a Bernoulli sample with mean $\gamma_{n_{\text{int}}}(\theta) = \Pr_{s\sim D}\big(\widehat{p}_{n_{\text{int}}}(\theta, s) < \tau_{\text{verify}}\big)$, i.e., the violation probability induced by the internal budget n_{int}. For increasing n_{int}, $\gamma_{n_{\text{int}}}(\theta)$ becomes a closer proxy of the true $\gamma(\theta) = \Pr_{s\sim D}\big(p(\theta, s) < \tau_{\text{verify}}\big)$.

After N samples, the empirical violation probability $\widehat{\gamma}_N = \frac{1}{N}\sum_{k=1}^{N} Z_k$ is computed together with its Wilson confidence interval $[\ell_\gamma, u_\gamma]$. As in the sequential

scenario-wise case, the number of samples is increased until the interval radius reaches a target accuracy or a maximum is reached.

Figure 5.5 shows the distribution of SMC win-rates observed in the four QG runs, while Figure 5.6 tracks the convergence of $\widehat{\gamma}_N$ and of its confidence interval.

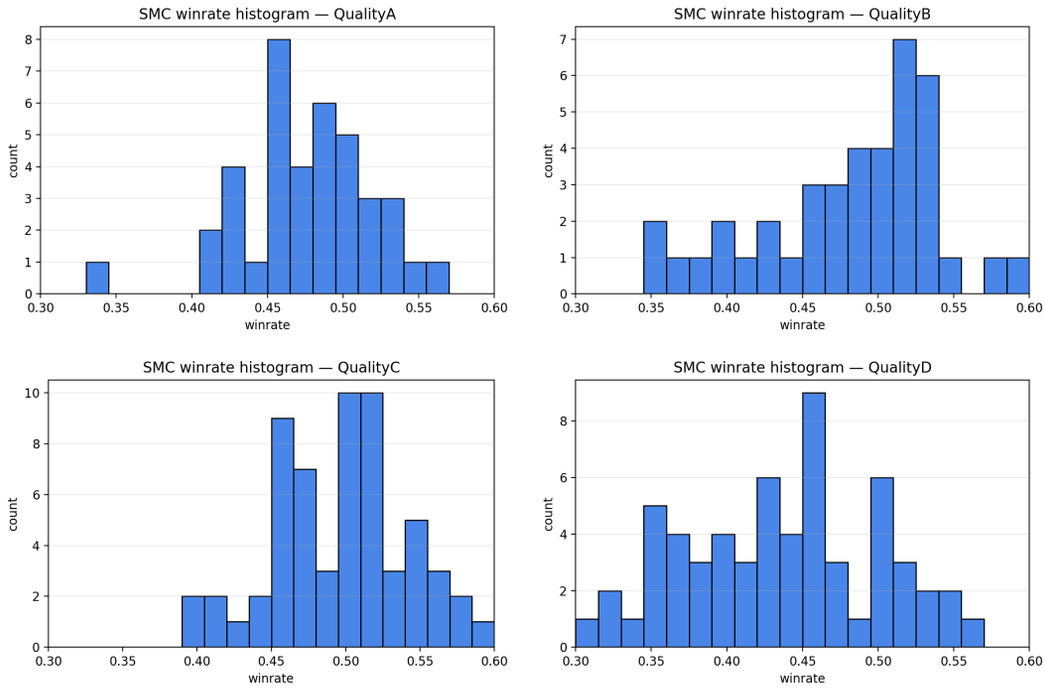

Figure 5.5. Histograms of SMC win-rates under distribution D for runs QualityA–D. In these runs, threshold $\tau_{\text{verify}} = 0.6$, yielding a violation indicator $Z_k = 1$ whenever the estimated win-rate $\widehat{p}_{n_{\text{int}}}(\theta, s^{(k)})$ falls below τ_{verify}.

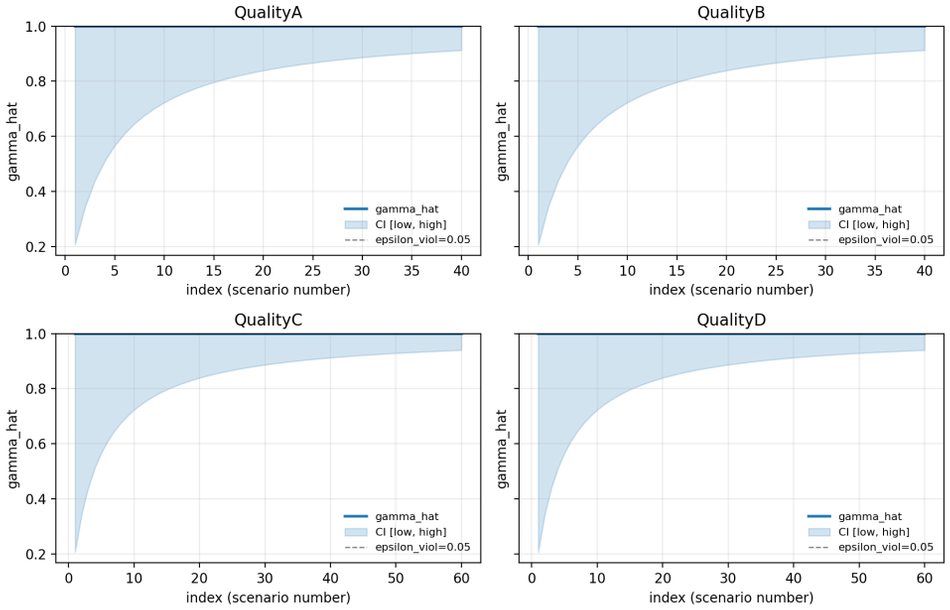

Figure 5.6. Convergence of the SMC estimate of the empirical violation probability $\gamma_{n_{\text{int}}}(\theta)$ as the number of sampled scenarios N increases. Although the horizontal axis is labeled as a scenario index in the plot, it corresponds to the sequential accumulation of SMC samples, i.e., to increasing values of N. Solid curves show the empirical estimate $\widehat{\gamma}_N$ for runs QualityA–D, shaded regions denote Wilson confidence intervals, and the horizontal dashed line marks a typical QG tolerance $\varepsilon_{\text{viol}} = 0.05$.

In all four QG runs used in the chess experiments, the estimated violation probability is $\widehat{\gamma}_N = 1.0$, meaning that every sampled scenario produced $\widehat{p}_{n_{\text{int}}}(\theta, s^{(k)}) < \tau_{\text{verify}}$ under the chosen internal budget n_{int}. As a consequence, the Wilson interval rapidly concentrates near 1 as N grows, as shown in Figure 5.6.

Table 5.2 reports the final SMC statistics for the four runs.

Run	N	Violations	$\hat{\gamma}$	CI low	CI high	Radius
`logs_runQualityA`	40	40	1.000	0.912	1.000	0.044
`logs_runQualityB`	40	40	1.000	0.912	1.000	0.044
`logs_runQualityC`	60	60	1.000	0.940	1.000	0.030
`logs_runQualityD`	60	60	1.000	0.940	1.000	0.030

Table 5.2. SMC global estimate summary. For each run, N is the number of SMC scenarios, $\hat{\gamma}$ is the empirical violation probability, and [CI low, CI high] is the final Wilson confidence interval.

In runs QualityA and B, the SMC procedure stops after $N = 40$ scenarios with a final interval $[0.912, 1.000]$, corresponding to a radius of about 0.044. In runs QualityC and D, a larger budget of $N = 60$ scenarios yields an even tighter interval $[0.940, 1.000]$ with radius 0.030. In all cases the lower bound is well above any reasonable QG tolerance $\varepsilon_{\text{viol}}$ (such as 0.01 or 0.05), and the SMC algorithm thus certifies that the empirical violation probability $\gamma_{n_{\text{int}}}(\theta)$ is essentially 1 for the chosen internal budget n_{int}. In terms of the QG stopping criterion $u_\gamma \leq \varepsilon_{\text{viol}}$, this means

that the loop never reaches an acceptance condition: with high confidence, the current controller is far from satisfying the requirement under the distribution D.

From the QG perspective this is a negative but highly informative result. It shows that, while ES and scenario-wise verification manage to push the training scenarios close to the must-win threshold, the resulting controller does not generalize to the broader distribution D: under realistic sampling of seeds and opponents, violations are ubiquitous. The methodology behaves exactly as intended: instead of giving a false sense of security, it refuses to certify a controller whose estimated violation probability is close to one.

5.4.2 Nature of the violating scenarios

The SMC procedure not only estimates $\gamma(\theta)$, but also collects violating scenarios as counterexamples. Inspection of a subset of these scenarios reveals that violations are rarely due to single-move blunders in otherwise trivial positions. Instead, they tend to cluster around structurally difficult situations, such as:

- openings that produce highly unbalanced material or king safety in a few moves;
- tactical patterns where a short, accurate sequence is required to avoid losing material;
- endgames where the controller must play many precise moves to convert a long-term advantage.

This qualitative pattern is consistent with the limitations of a shallow, search-free neural controller: without a look-ahead mechanism, certain tactical traps simply cannot be detected from static evaluation alone. The fact that SMC finds such counterexamples is a feature, not a bug: it shows that the QG loop is able to surface the most problematic regions of the scenario space instead of hiding them behind aggregate statistics.

5.5 Overall Simulation Budget

Table 5.3 summarizes the number of simulated games used by each component of the QG loop across the four QG runs and the additional ES-only run `logs_runRich2`. Figure 5.7 provides a visual breakdown of the same data.

Run	ES games	Verification games	SMC games	Total
`logs_runQualityA`	800	400	400	1600
`logs_runQualityB`	800	400	400	1600
`logs_runQualityC`	2880	600	600	4080
`logs_runQualityD`	1728	600	600	2928
`logs_runRich2`	7500	750	750	9000
Grand total	13708	2750	2750	19208

Table 5.3. Number of games per run and per phase (ES, sequential verification, SMC). The last row reports the totals across all five runs.

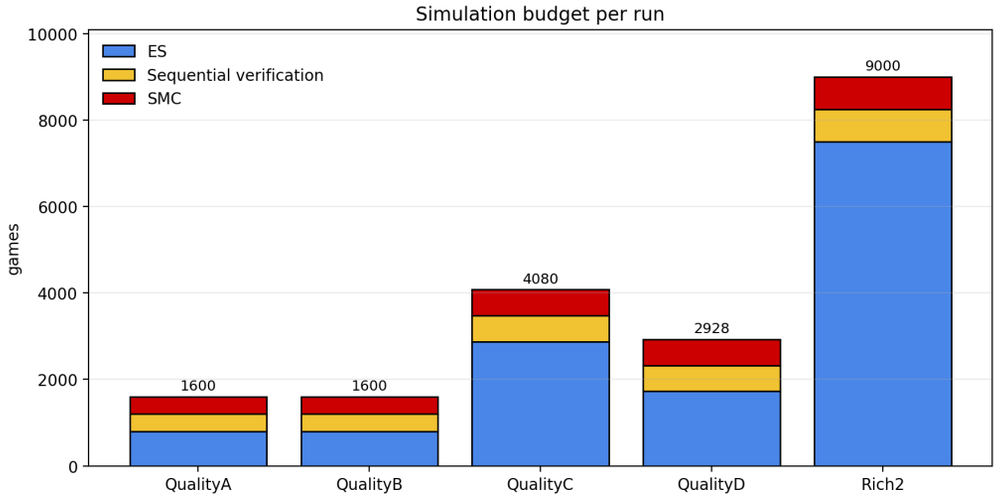

Figure 5.7. Stacked bar plot showing, for each run, the number of simulated games spent in ES, sequential verification and SMC. The labels on top of the bars show the total number of games per run.

Overall, the experiments required 19 208 simulated games. About 71.3% of these (13 708 games) are consumed by ES, reflecting the black-box nature of the optimization step. Scenario-wise sequential verification and global SMC each account for 2 750 games, i.e. about 14.3% of the total budget. This confirms that the statistical components of the QG loop add a relatively modest overhead on top of the optimization effort, while providing strong information about both scenario-wise and global behavior.

The richer ES-only run `logs_runRich2` uses 7 500 games for optimization and a moderate number of additional games for verification and SMC-style evaluation, illustrating how the same infrastructure can scale to larger optimization budgets without changing the statistical machinery.

5.6 Match-level behavior and Baseline Comparison

Beyond aggregate win-rates and violation probabilities, it is instructive to inspect how the trained controller behaves at the level of individual games and how it

compares to the baseline opponents.

To this end, an early controller snapshot (after a limited number of ES iterations) and a late controller snapshot (after ES, verification and SMC) are evaluated in head-to-head matches against four opponents: `Random-Player`, `Greedy-Material-Player`, `PSQT-Greedy-Player` and `Minimax-Player`.

For each pairing, a large batch of games is played and the empirical outcome frequencies and median game length are recorded.

5.6.1 Games against simple opponents

Against the random opponent, the controller exhibits the expected behavior of a reasonably trained policy. Table 5.4 shows that the early controller already wins 63.2% of games, drawing 10.3% and losing 26.5%, with a median game length of 180 moves (plies counted as half-moves in the logs). The late controller further improves the win-rate to 73.6%, reduces the draw rate to 3.5% and the loss rate to 22.9%, and closes games more quickly, with a median of 150 moves.

These numbers indicate that ES and the QG loop manage to produce a controller that can reliably exploit the weaknesses of a purely random policy and convert advantages in a reasonably decisive fashion. The reduction in median game length suggests that the late controller is more efficient at capitalizing on tactical opportunities once they appear.

5.6.2 Games against stronger baselines

The picture changes markedly when moving to stronger baselines. Against `Greedy-Material-Player` and `Minimax-Player`, both early and late controllers fail to score a single win: in all cases, the empirical win-rate is 0%, while draws account for around 83% of games and losses for 15%–18%, with median game lengths of around 72–77 moves. Against `PSQT-Greedy-Player` the situation is even more challenging: the controller again never wins, and the loss rate increases from 26.0% (early) to 39.8% (late), with very long median game lengths around 112–114 moves.

This behavior reflects the limitations of a shallow, search-free neural controller. Against opponents that exploit even a small amount of look-ahead or a more refined static evaluation (as in the PSQT case), the network can often maintain an approximately balanced position for many moves but struggles to convert slight advantages into wins. The late controller is more decisive against random play but does not consistently manage to break through the defence of search-based baselines.

5.6.3 Quantitative match statistics

Table 5.4 summarizes the match-level results by reporting, for each opponent, the empirical outcome frequencies and median game length for the early and late controllers.

Opponent	Early W	Early D	Early L	Late W	Late D	Late L
`Greedy-Material-Player`	0.0%	82.8%	17.2%	0.0%	82.1%	17.9%
`Minimax-Player`	0.0%	83.9%	16.1%	0.0%	85.1%	14.9%
`PSQT-Greedy-Player`	0.0%	74.0%	26.0%	0.0%	60.2%	39.8%
`Random-Player`	63.2%	10.3%	26.5%	73.6%	3.5%	22.9%

Table 5.4. Early vs. late match outcomes by opponent. Win, draw and loss percentages are computed over the same number of games for each controller–opponent pairing.

Comparing early and late versions of the controller reveals a clear qualitative effect of the QG loop. Against the random baseline, the controller becomes both stronger and more decisive: the win-rate increases by more than 10 percentage points, draws become rare, and games tend to finish earlier. This matches the intuition that feasibility-oriented ES training successfully teaches the network to exploit obvious tactical mistakes and to convert material advantages more reliably.

Against search-based baselines, instead, the story is different. The late controller does not manage to turn drawn positions into wins and, in the case of `PSQT-Greedy-Player`, even loses more often than the early snapshot. From the QG viewpoint this is perfectly consistent with the SMC verdict (Table 5.2): the controller may look competent in many games, but the space of realistic scenarios contains a large fraction of positions where static evaluation alone is not enough to avoid violations of the must-win requirement.

Overall, the match-level analysis confirms that the QG loop is informative beyond aggregate win-rates: it exposes, in concrete game statistics, the gap between "playing plausibly" and "provably satisfying the specification".

5.7 Summary of Experimental Findings

The experiments reported in this chapter address the main questions posed in the introduction of the thesis and illustrate both the strengths and the current limitations of the QG loop when applied to a non-trivial game such as chess.

1. **Feasibility-oriented training drives systematic improvement but does not fully meet the specification.** ES, when coupled with the feasibility loss $J_S(\theta)$, is able to find neural controllers that substantially improve over the initial policy and achieve median scenario win-rates between 43% and 49% at threshold $\tau = 0.5$, as seen in Figures 5.1 and 5.2. However, even the best run leaves some scenarios below the must-win requirement, and no run achieves a fully feasible controller on the training set.

2. **Sequential verification provides explicit scenario-wise uncertainty quantification.** The Wilson-interval-based verifier concentrates samples on the most ambiguous scenarios and yields per-scenario confidence intervals (Table 5.1, Figures 5.3 and 5.4). In the present case study all scenarios end up borderline at $\tau = 0.6$, indicating that empirical win-rates in the range 0.59–0.70 are not sufficient, given the available budget, to prove or disprove the must-win requirement. This is a negative outcome from the guarantee

viewpoint, but a positive one from a modeling viewpoint: it turns the informal notion of "almost good enough" into a quantitative statement.

3. **SMC exposes the lack of global robustness.** By sampling scenarios from D and estimating $\gamma(\theta)$ with absolute-error control, SMC bridges the gap between the finite training set and the broader scenario space that the controller may encounter. In all QG runs the estimated violation probability converges to $\hat{\gamma} = 1.0$ with tight Wilson intervals such as $[0.940, 1.000]$ (Table 5.2). This shows that the current controller is far from satisfying any realistic QG tolerance and that further architectural changes or training strategies are needed to obtain non-trivial guarantees.

4. **The simulation costs remain moderate and transparently accounted for.** The entire experimental campaign uses 19 208 simulated games, of which about 71% are devoted to ES and the remaining 29% are split evenly between sequential verification and SMC (Table 5.3, Figure 5.7). This confirms that the statistical machinery of the QG loop can be deployed with a modest overhead on top of existing black-box optimization pipelines.

5. **The resulting controller plays plausible chess but is clearly weaker than simple search-based baselines.** Match-level experiments show that the controller decisively outperforms a random opponent and improves its win-rate and decisiveness after QG refinement, but fails to score wins against even modest search-based baselines and loses a significant fraction of games (Table 5.4). This is consistent with the negative SMC verdict and emphasizes that, in its current form, the controller should not be deployed in safety-critical settings without further refinement.

Taken together, these findings validate the QG framework as a practical way to interrogate learning-based controllers: even when the final verdict is negative (i.e. the requirements are not met), the combination of ES, sequential verification and SMC produces a detailed, statistically grounded picture of where and how the controller fails, and at what computational cost. In future work, the same methodology could be applied to stronger network architectures and richer opponent models, with the ultimate goal of turning informal intuitions about "playing well" into explicit quality guarantees.

Chapter 6

Conclusions

This thesis has investigated the simulation-based design of intelligent systems with explicit quality guarantees, instantiating the Quality-Guaranteed (QG) learning loop on a non-trivial adversarial domain: chess. The proposed pipeline combines Evolution Strategies (ES), sequential scenario-wise verification, Statistical Model Checking (SMC), and counterexample-guided refinement into a single closed loop that optimizes a neural controller while simultaneously estimating its probability of violating a must-win requirement.

The chapter summarizes the main findings, answers the research questions introduced in Chapter 1, discusses the limitations of the current case study, and outlines directions for future work.

6.1 Summary of the Thesis

The work starts from the observation that most machine–learning pipelines optimize for average performance but provide little support for quantitative guarantees such as "the system wins with probability at least τ and confidence $1 - \delta$". To bridge this gap, the thesis adopts the QG paradigm: learning is guided by a requirement and is tightly coupled with statistical verification.

From a methodological point of view, the thesis:

- formalizes the chess environment as a controlled simulation model with reproducible scenarios, built on top of the `GameState` engine;
- defines a lightweight feed-forward neural controller $f_\theta(x)$ and a requirement in terms of win-rate thresholds over a distribution of scenarios;
- tailors ES to feasibility-oriented training, where the objective is to minimize a loss that penalizes violation of the win-rate requirement;
- develops a sequential verifier that, for each scenario, adapts the number of simulated games and returns Wilson confidence intervals and pass/borderline/fail decisions at a prescribed confidence level;
- integrates a global SMC procedure that estimates the violation probability $\gamma(\theta)$ under a distribution of unseen scenarios, again with Wilson-type confidence intervals and explicit error control;

- implements the full QG loop in Python, including scenario and counterexample management, logging, and tools for match-level analysis.

Experimentally, the thesis applies this pipeline to a family of QG runs in which the controller is trained and refined against a small, fixed set of scenarios and then evaluated by SMC on a broader scenario distribution. The analysis in Chapter 5 quantifies:

- how ES improves median and worst-case win-rates on the training set without fully satisfying the must-win requirement;
- how sequential verification concentrates its effort on statistically ambiguous scenarios and provides per-scenario confidence intervals;
- how SMC reveals that, despite local improvements, the global violation probability is essentially 1, exposing a lack of robustness outside the training set;
- how the overall simulation budget is distributed across ES, verification and SMC, and how this budget remains moderate for the scale of the case study;
- how the trained controller behaves at match level against random and search-based opponents, highlighting the gap between "plausible" play and true satisfaction of the requirement.

6.2 Answers to the Research Questions

RQ1. Can a lightweight neural controller trained via ES satisfy a must-win requirement across a finite set of scenarios?

The experiments show that feasibility-oriented ES training is able to substantially improve the controller on the finite scenario set. Across the QG runs, median scenario win-rates approach the training threshold and the fraction of scenarios whose empirical win-rate exceeds the threshold increases over time. However, no run achieves a controller that satisfies the must-win requirement on all training scenarios.

In other words, ES is effective at pushing the controller towards feasibility, but the chosen architecture and budget are not sufficient to cross the boundary between "almost good enough" and provably satisfying the requirement. From the control-design perspective, this indicates that feasibility-driven ES can serve as a powerful optimization backbone, but its success ultimately depends on the expressive power of the controller class and on the available simulation budget.

RQ2. Can sequential verification and SMC provide meaningful guarantees within realistic simulation budgets?

Sequential verification and SMC prove to be both informative and efficient. On the training scenarios, the sequential verifier adaptively selects sample sizes up to a fixed cap per scenario and returns Wilson intervals whose width is commensurate with the budget. In the chess case study, the verifier classifies that all scenarios are statistically

borderline with respect to the must-win threshold: empirical win-rates are close to the threshold, but the confidence intervals do not allow a definitive pass/fail verdict. This negative outcome is nevertheless valuable, as it turns qualitative statements such as "the controller seems to win most of the time" into precise bounds on the underlying probabilities.

At the global level, SMC estimates the violation probability $\gamma(\theta)$ under a scenario distribution that samples seeds and opponents. For all QG runs, the SMC procedure produces narrow Wilson intervals, whose lower bound is close to 1, showing that violations are ubiquitous under realistic sampling. Despite this pessimistic verdict, the required number of simulated games remains modest compared to the ES budget, and the entire pipeline runs within a few tens of thousands of games. This confirms that, at least for simulation domains of similar complexity to chess, QG-style guarantees can be obtained within realistic computational resources.

RQ3. Does counterexample-guided refinement improve robustness to unseen scenarios?

The counterexample mechanism collects failing scenarios from sequential verification and SMC and feeds them back into the training set. In principle, this process should focus ES on the most problematic regions of the scenario space and reduce the estimated violation probability over successive QG iterations.

In the present case study, the results are mixed. On the one hand, the added scenarios do enrich the training set and make the feasibility loss more demanding, as evidenced by the evolution of training win-rates across runs. On the other hand, SMC consistently reports a violation probability close to 1, indicating that, given the current controller class and budget, the QG loop is unable to drive the system into a genuinely low-violation regime.

The main lesson is that counterexample-guided refinement is effective at *exposing* failure modes—it surfaces structurally difficult situations such as sharp tactical traps and long forcing endgames—but it cannot by itself compensate for insufficient model capacity or for an overly ambitious requirement. In this sense, the QG loop behaves as an adaptive diagnostic tool: when it fails, it fails with explanations.

6.3 Limitations of the Case Study

The conclusions above are conditioned on several modeling choices and simplifications.

Controller architecture. The neural controller is intentionally shallow and compact, chosen for speed and interpretability rather than for raw playing strength. This makes it easier to attribute failures to the lack of look-ahead or representational power, but also limits the attainable performance. The negative SMC verdict should therefore not be interpreted as a fundamental limitation of QG control, but as a consequence of this design choice.

Scenario distribution and requirement. The requirement is defined over a specific distribution of scenarios that mixes different opponents and pseudo-random

seeds. Guarantees, as well as violation probabilities, are meaningful only with respect to this distribution. Changing the mix of opponents, the time controls, or the sampling of seeds would lead to different numerical conclusions. Moreover, the chosen must-win threshold and tolerance are intentionally strict: in many practical applications, weaker requirements (for example, on expected regret or on risk-sensitive utilities) might be more appropriate.

Scale and computational budget. The total number of simulated games, while non-negligible, is small compared to modern reinforcement-learning pipelines in board games. The case study is therefore best seen as a proof of concept on a constrained budget, not as an attempt to compete with state-of-the-art chess engines. Scaling QG methods to larger controllers and richer scenario spaces will require additional engineering effort, parallelization, and careful tuning of stopping criteria.

Domain specificity. Chess is deterministic and fully observable, with a well-defined simulator and no real-world safety implications. These properties make it an ideal testbed for methodological work, but they also remove several challenges that arise in cyber-physical systems, such as model uncertainty, sensor noise, or mixed continuous–discrete dynamics. Extrapolating the quantitative results of this thesis to such domains would be unjustified; what carries over is the *structure* of the QG loop.

6.4 Perspectives and Future Work

The experimental evidence collected in this thesis suggests several directions for further research.

Stronger controllers and richer opponents

A natural extension is to increase the expressive power of the controller, for example by using deeper networks, convolutional architectures, or separate policy and value heads. Coupling such controllers with search (e.g. a Monte Carlo tree search that uses f_θ as evaluation function) would also test how QG methods interact with look-ahead mechanisms. On the opponent side, introducing stronger baselines and more diverse playing styles would make the scenario distribution more challenging and provide a finer-grained picture of robustness.

Alternative requirements and multi-objective control design

The current requirement is defined as a simple must-win constraint on win-rate. In many applications, one would like to trade off mean performance, risk of catastrophic failure, and computational cost. Extending the QG framework to multi-objective or hierarchical requirements—for example by combining constraints on win-rate, maximum loss probability, and resource usage—would bring it closer to real engineering practice. From a statistical point of view, this would require joint confidence statements over several metrics.

Improved statistical machinery

On the verification side, there is room for more advanced statistical methods. Sequential tests with tighter stopping rules, variance reduction techniques, or Bayesian credible intervals could reduce the number of required simulations while preserving rigorous guarantees. Adaptive allocation of the simulation budget across scenarios, based on their estimated difficulty, could further increase efficiency. Exploring these options in combination with ES and counterexample refinement is a promising line of work.

Applications beyond chess

Perhaps the most important perspective is to transfer the lessons of this case study to domains where quantitative guarantees are directly tied to safety or reliability. Examples include motion planning for autonomous vehicles, resource management in communication networks, or supervisory control in industrial automation. In such settings, QG control could serve as a bridge between data-driven learning and traditional verification, providing both high-performing controllers and explicit upper bounds on failure probability.

Closing Remarks

The chess case study demonstrates that a Quality-Guaranteed learning loop can be instantiated end-to-end on a complex, adversarial domain, yielding not only a trained neural controller but also a rich set of statistical diagnostics about its behavior. The final verdict in this particular experiment is negative: the controller does not satisfy the must-win requirement under the chosen scenario distribution. Yet this negative result is itself informative, because it is supported by explicit confidence bounds and accompanied by concrete counterexamples.

In this sense, the main contribution of the thesis is conceptual rather than competitive: it shows that learning and verification need not be separate phases, and that simulation-based optimization can be embedded in a loop that continuously tests, explains, and, when possible, guarantees the behavior of an intelligent system.

Acknowledgements

I would first like to thank myself for having brought this journey to completion. For the patience, for the strength with which I faced the mountains encountered along the way, and for the courage to forgive my mistakes, recognizing them as part of who I have been and who I will become. I am grateful to my past self for choosing persistence over comfort, and I hope my future self will look back at this moment with the same determination and quiet pride. I thank myself for not giving up in the moments when it would have been easier to stop, for continuing to study, to improve, and to believe that this goal was within my reach.

I would like to thank my family, who have supported me not only throughout this university path, but since the very first day of my life. They gave me the opportunity to receive a proper education and, even more importantly, helped me understand how essential knowledge is in life.
Among all the people, I wish to dedicate a few lines to my mother, Cornelia Banua Trinidad: without her, I would not be the person I am today. She has been my source of light in every dark moment and allowed me to cultivate my passions, while at the same time giving me the freedom to explore the world in my own way. I thank her for her silent sacrifices, for believing in my future even when I myself had doubts, and for teaching me, by example, what it truly means to commit oneself to those we love.

A special thanks also goes to my primary school teachers, Maria Grazia Bultrini and Francesca Diaco, who guided me through the very first steps of my academic journey and believed in me from the beginning. I owe them my foundations, but also something even more valuable: curiosity for learning, the patience to try again after making mistakes, and the awareness that, with perseverance and commitment, one can truly learn anything.

A thought of gratitude goes to my closest friends — Rita Rotondaro, Greta Quarta, Aurora Parlato, Laura Stefani, Federico Rizzo, Matteo Greco, Alfonso De Cesare and Luca Carioscia — who have accompanied me on this journey. Thank you for sharing with me both the lighter days and the difficult ones, for the laughter, the support, and your constant presence. Each of you has made this path less heavy and infinitely more meaningful.

A special word of thanks also goes to my university colleagues — Alberto Pulcini, Daniele Marchetilli, Daniele Spadea, and Bernardo Marchese — who shared with me not only lectures and exams, but also doubts, ambitions, and countless conversations. Growing alongside you made this journey intellectually stimulating and personally enriching. Thank you for the discussions, the mutual support, and for turning challenges into shared experiences.

I would also like to thank God, if He exists: I like to think that, somewhere, He is smiling. I hope that, if He truly is watching, He might be at least a little proud of the path I am walking, and that He will continue to give me the clarity and strength I need to pursue my dreams.

This thesis is not a conclusion, but a beginning. It is a reminder that growth often comes quietly — through effort repeated, doubts faced, and small steps taken consistently over time. It represents not only what I have learned so far, but the responsibility to keep learning, to keep questioning, and to build systems — and a life — that reflect the values I choose to stand for. Whatever comes next, I move forward with gratitude, humility, and the quiet determination that brought me here.

And to the beginning of this path — thank you for daring to start.

Bibliography

[1] Petr Bulychev, Alexandre David, Kim G. Larsen, Axel Legay, Marius Mikucionis, Danny Bøgsted Poulsen, and Zheng Wang. Uppaal-smc: Statistical model checking for priced timed automata. In *Proceedings of the 10th Workshop on Quantitative Aspects of Programming Languages and Systems (QAPL 2012)*, volume 85 of *Electronic Proceedings in Theoretical Computer Science*, pages 1–16. Open Publishing Association, 2012.

[2] Alexandre David, Dehui Du, Kim G. Larsen, Axel Legay, Marius Mikucionis, Danny Bøgsted Poulsen, and Sean Sedwards. Statistical model checking for stochastic hybrid systems. In *Proceedings of the 1st International Workshop on Hybrid Systems and Biology (HSB 2012)*, volume 92 of *Electronic Proceedings in Theoretical Computer Science*, pages 122–136. Open Publishing Association, 2012.

[3] Marco Esposito, Alberto Leva, Toni Mancini, Leonardo Picchiami, and Enrico Tronci. Simulation-based design of industry-size control systems with formal quality guarantees. *IEEE Transactions on Industrial Informatics*, 21(5):3871–3879, 2025.

[4] Niklas Fiekas. python-chess: A chess library for Python. https://python-chess.readthedocs.io/, 2024.

[5] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975.

[6] Ingo Rechenberg. *Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution.* Frommann-Holzboog, Stuttgart, 1973.

[7] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.

[8] Hans-Paul Schwefel. *Numerische Optimierung von Computer-Modellen mittels der Evolutionsstrategie.* Birkhäuser, Basel, 1977. In German.

[9] Koushik Sen, Mahesh Viswanathan, and Gul Agha. Statistical model checking of black-box probabilistic systems. In *Computer Aided Verification (CAV 2005)*, volume 3576 of *Lecture Notes in Computer Science*, pages 202–215. Springer, 2005.

[10] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

[11] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008.

[12] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[13] Abraham Wald. *Sequential Analysis*. Wiley, 1947.

[14] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[15] Håkan L. S. Younes. Probabilistic verification of discrete event systems using acceptance sampling. In *Computer Aided Verification (CAV 2004)*, volume 3114 of *Lecture Notes in Computer Science*, pages 223–235. Springer, 2004.